Patrones de Diseño

© JMA 2020. All rights reserved

PROGRAMACIÓN ORIENTADA A OBJETOS

© JMA 2001. All rights reserved

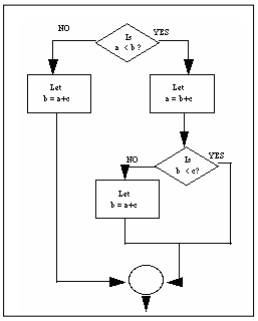
Un poco de Historia

- La programación orientada a objetos es una de las formas más populares de programar actualmente con gran acogida por parte de los programadores.
- Esta gran aceptación es debido a sus grandes capacidades y ventajas frente a las antiguas formas de programas.
- La POO ofrece amplias mejoras en el diseño, desarrollo y mantenimiento de Software ofreciendo soluciones de portabilidad y reusabilidad.

© JMA 2001. All rights reserved

Programación Lineal

- Tradicionalmente, la programación fue hecha en una manera secuencial o lineal, es decir, una serie de pasos consecutivos con estructuras consecutivas y bifurcaciones.
- Los lenguajes basado en esta forma de programación estaban muy bien, hasta que se volvían complejos.
- El mayor problema de la programación “espagueti” es que no ofrecían flexibilidad ni mantenimiento.



© JMA 2001. All rights reserved

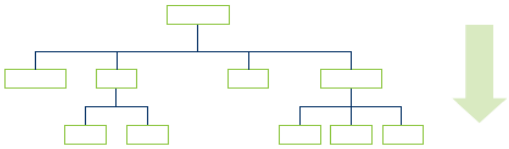
Programación Estructurada

- Frente a esta dificultad, surgieron los lenguajes basados en la programación estructurada. (C, Pascal, Modula,…)
- Su idea principal es la de hacer una programación separada en partes (módulos) se sean ejecutados conforme se requieran.
- Aparece un diseño modular, compuesto por módulos independientes.
- Esta programación se orienta a intentar descomponer el programa mas y mas.

© JMA 2001. All rights reserved

Programación Estructurada

- Según indicó Edsger Dijkstra , todo programa se puede escribir usando:
  - 3 estructuras básicas de control:
    - Secuenciales, Condicionales, Iteraciones
  - No utilización de la sentencia GOTO
  - Definición de Módulos
- De esta forma nos encontramos un diseño TOP-DOWN (esta división toma la forma de un árbol cuya raíz es el programa principal)



© JMA 2001. All rights reserved

Programación Estructurada

- El problema que tuvo esta programación, es la creación de programas mas y mas grandes y complejos.
- El mantenimiento de todos los módulos, junto con la imposibilidad de su reutilización y efectos colaterales, fue la consecuencia de la pérdida de fuerza en el horizonte de las técnicas de programación.
- De esta forma, aparece la Programación Orientada a Objetos como una evolución de la Programación Estructurada.

© JMA 2001. All rights reserved

Programación Orientada a Objetos

- El primer enfoque hacia la Orientación a Objetos, se produjo a finales de los años 70. Sin embargo hasta los años 90 no tuvieron una evolución final.
- La necesidad de programas cada vez mas grandes y complejos, así como la necesidad de reutilización, ayudo a ello.
- POO surge como una evolución de la Programación Estructurada, añadiendo nuevos conceptos y formas de hacer las cosas.
- POO se basa en dividir el programa en pequeñas unidades lógicas de código (objetos)

© JMA 2001. All rights reserved

Programación Orientada a Objetos

- Los objetos son unidades independientes que se comunican mediante mensajes.
- Las características de POO los podríamos resumir en:

La Programación Orientada a Objetos ayuda a los desarrolladores a reflejar en papel el mundo real.

© JMA 2001. All rights reserved

Programación Orientada a Objetos

- Características
  - Relaciona el sistema con la Vida Real.
  - Permite su aplicación a Sistemas Complejos.
  - Reutilización y Extensión del Código
  - Facilita creación de programas Visuales.
  - Agiliza el desarrollo
  - Facilita trabajo en equipo
  - Facilita Mantenimiento de Sotfware.

- Un punto a tener muy en cuenta es que POO proporciona conceptos claros y Herramientas con las cuales se modela y representa el mundo real.

© JMA 2001. All rights reserved

Fundamentos

- La base fundamental es intentar llevar el problema real al papel mediante objetos.
- Para entender la POO necesitamos conocer una serie de conceptos claves y como interactúan entre ello.
- Conceptos como: objetos, clases, abstracción, etc, forman parte del mundo de la programación Orienta a Objetos.

© JMA 2001. All rights reserved

Objetos

- Son la clave de los lenguajes Orientados a Objeto.
- Su mejor definición, es definir un OBJETO en el mundo real.
- Definición Objeto Real:
  - Es cualquier cosa que vemos a nuestro alrededor (teléfono, árbol, coche, ordenador, etc)
  - En los objetos del mundo real, no necesitamos ser un experto para usarlo, ni como funciona al 100%. Por ejemplo:
    - ordenador (placa, procesador, HD, etc..)
    - árbol (tipo fruta, dame fruta, regarlo, etc…)

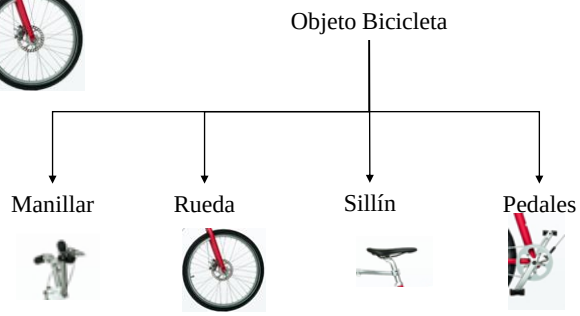
© JMA 2001. All rights reserved

Objetos

- Cada objeto en el mundo real, puede estar compuesto por componentes internos (objetos) pero trabajan en conjunto y son autónomos.
- La Programación Orientada a Objetos trabaja de esa misma forma.
- Los objetos no son componentes aislados, sino que forman parte de una jerarquía o de otros objetos

© JMA 2001. All rights reserved

Objetos



© JMA 2001. All rights reserved

Objetos

- Todos los objetos reales tiene una serie de características y elementos que los diferencia de los demás, esto es, tienen:

	Reales	POO
De un Coche podemos decir:	marca, modelo, color	atributos
Podemos hacer estas cosas:	frenar, adelantar	métodos

- En Programación, también se define un objeto como una instancia de una clase.

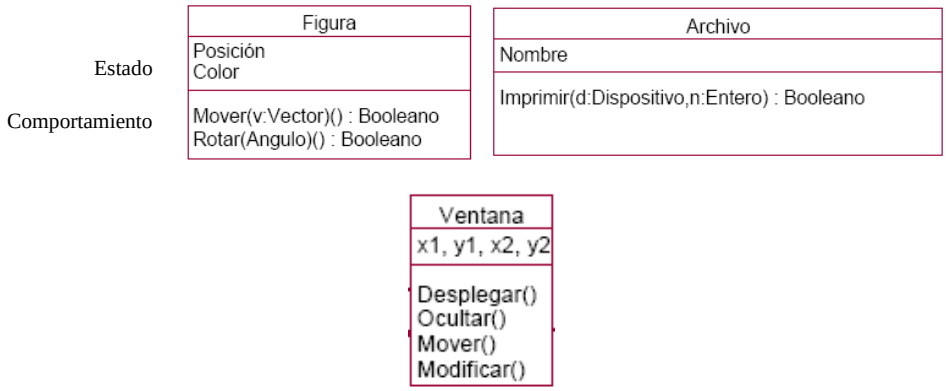
© JMA 2001. All rights reserved

Componentes

- Los objetos están compuestos de :
  - Características (Propiedades/Estado)
    - Son los diferentes atributos (variables) que diferencian un objeto de otro
  - Comportamiento (implementación).
    - Es el código que logra el funcionamiento del objeto, métodos (también llamados funciones, subrutinas, etc)
  - Interface
    - Conjunto de métodos, propiedades, eventos y atributos que permiten la interconexión con el exterior
- La representación de los objetos se puede hacer usando diferentes METODOLOGÍAS, normalmente UML o también MLG.

© JMA 2001. All rights reserved

Componentes



© JMA 2001. All rights reserved

Atributos

- También conocidos como campos, son la memoria del objetos.
- Todos los objetos definen su estado con sus atributos.
- Se corresponden con las clásicas variables de cualquier programa, pero con diferencias: los atributos se están asociados al objeto (son parte de el), se pueden heredar, las variables no.
- Se definen dentro del propio objeto.
- Estos atributos pueden estar disponibles por definición propia o por herencia.

© JMA 2001. All rights reserved

Métodos

- Los métodos son el comportamiento del objeto: fragmentos procedurales, asociados al objeto, que acceden a los datos del objeto y sólo se pueden usar a través de mensajes (llamadas).
- Los métodos en POO son sinónimos de otros elementos de otras metodologías de programación como: procedimientos, funciones, rutinas, etc.
- Los métodos pueden tener argumentos y pueden estar disponibles por definición propia o por herencia.

© JMA 2001. All rights reserved

Clases

- Al igual que ocurre en el mundo real, existen multitud de objetos del mismo tipo (coches, frigoríficos, cajeros, etc) todos ellos independientes, distintos pero con las misma base de creación.

© JMA 2001. All rights reserved

Clases

- Una clase es un prototipo, modelo o plantilla que define los atributos y métodos comunes para poder crear objetos del mismo tipo: las instancias.
- Las clases definen las características y el comportamiento que luego las instancias utilizarán.
  - CLASES → Modelo/plantilla (diseño)
  - INTANCIAS → Casos concretos (ejecución)
- Tanto las clases como las instancias son objetos.

© JMA 2001. All rights reserved

Clases

Clase Teléfono
Marca Memoria
Llamar() Colgar ()

Objeto 1
Nokia 1 Gb
Llamar() Colgar()

Clase Bicicleta
Material Modelo
Pedalear() Freanar()

Objeto 1
Madera Futuro
Pedalear() Frenar()

© JMA 2001. All rights reserved

Clases

- Para poder crear un objeto, es necesario disponer de la clase de la que vamos a instanciar el objeto.
- Cuando definimos un objeto de una clase determinada, los objetos comparten su COMPORTAMIENTO (métodos) pero tendrán valores diferentes en sus PROPIEDADES (Atributos).
- En programación, crear una clase dentro de una aplicación, implica crear un nuevo tipo de dato en la aplicación.

© JMA 2001. All rights reserved

Tipos de Operaciones

- Dentro de lo POO disponemos de una serie de operaciones que nos permiten gestionar el objeto.
- Estás operaciones serán implementadas de alguna forma dependiendo del lenguaje de programación.
- La forma de implementación es indiferente, sólo nos preocupa su funcionalidad.
  - Constructor — Crear una nueva instancia de la Clase (objeto)
  - Destructor — Destruye una instancia concreta.
  - Selector — Selecciona una parte del Estado o atributo de un objeto.
  - Modificador — Modifica una parte del Estado o atributo.
  - Clonador — Copia una instancia en otra nueva.
  - Iterador — Permite recorrer una estructura de datos (objetos).
  - Visualizador — Muestra todo el estado de un objeto de forma elaborada.

© JMA 2001. All rights reserved

Constructores y destructores

- Los constructores dirigen la creación del objeto, inicializan o asignan valor a los atributos y realizan las operaciones necesarias para crear una instancia plenamente funcional con un estado controlado.
- Un error en el constructor cancela la creación de la instancia.
- Los destructores dirigen la desaparición del objeto, permiten controlar como se liberan los recursos del objeto cuando el sistema destruye el objeto.
- Los destructores son invocados automáticamente por el sistema y son únicos por clase.

© JMA 2001. All rights reserved

Propiedades

- Las propiedades pertenecen a la orientación a componentes pero muchos lenguajes orientados a objetos permiten su implementación.
- Las propiedades se comportan como atributos cuando se obtiene acceso a ellas. Pero, a diferencia de los atributos, las propiedades se implementan con métodos de acceso que definen las instrucciones que se ejecutan cuando se recupera o se asigna una propiedad. Las propiedades permiten la comodidad de los atributos pero con la robustez de los métodos.
- No existe necesariamente una correspondencia uno a uno entre atributos y propiedades, aunque en la mayoría de los casos se produce, una propiedad puede ser la síntesis de varios atributos o una parte de un atributo.

© JMA 2001. All rights reserved

Miembros de clase

- Dado que la clases son objetos pueden tener atributos y métodos de la propia clase independientemente de las instancias de la clase. Por lo tanto los miembros se clasifican en miembros de clase (o estáticos) y miembros de instancia.
- Los atributos de clase son únicos por clase, se comparten para todas las instancias de la clase y son accesibles desde cualquier método de clase o de instancia.
- Los métodos de clase solo pueden acceder a atributos y métodos de clase, dado que las instancias son múltiples para una clase o puede no existir ninguna. Los métodos de clase son accesibles a través del nombre de la clase.
- Los métodos de instancia tienen acceso a todos los miembros de clase.

© JMA 2001. All rights reserved

Principios de P.O.O

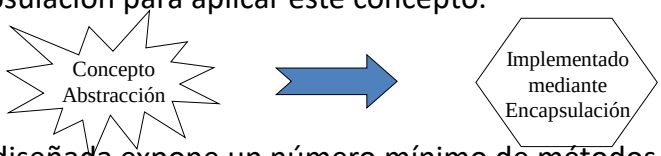
- Además de todo lo visto anteriormente, y según Booch: “si alguno de los siguientes elementos no existen, se dice que el modelo no es orientado a objetos”

- Estos elementos son:
  - Abstracción / Ocultación
  - Encapsulación
  - Modularidad
  - Jerarquía
  - Polimorfismo

© JMA 2001. All rights reserved

Abstracción

- Consiste en captar las características esenciales de un objeto, así como su comportamiento.
- Una buena abstracción se centra en la vista externa del objeto, eliminando todos los detalles pocos importantes.
- La Abstracción es un concepto teórico, y los lenguajes Orientados a Objeto utilizan la encapsulación para aplicar este concepto.



- Una clase bien diseñada expone un número mínimo de métodos cuidadosamente elegidos para proporcionar el comportamiento esencial de la clase.
- Hacer buenas abstracciones no es una tarea fácil.

© JMA 2001. All rights reserved

Encapsulación

- Es la propiedad que permite asegurar que el contenido de la información de su objeto está oculta al mundo exterior.
- La encapsulación también es conocido como OCULTACION DE INFORMACION.
- Gracias a ella, se nos permite la división de un programa en módulos.
- El encapsulamiento proporciona la abstracción y se implementa mediante clases.

© JMA 2001. All rights reserved

Encapsulación

- Si todo lo relacionado con el objeto estuviese oculto, no se podría acceder a él; en la práctica, existen 2 partes bien diferenciadas:
  - Interface
    - Es la parte externa del Objeto
  - Implementación
    - Es el código interno.
- Gracias a la encapsulación se controla el acceso a los datos y métodos, mediante modificadores de acceso en los lenguajes O.O
  - Private, Protected, Public, etc.

© JMA 2001. All rights reserved

Encapsulación

- ¿Por qué Encapsulación?
  - Es muy sencillo y responde a 2 importantes razones:
    - Al ocultar los detalles de la implementación se evita que se usen de manera no deseada y que se accedan de forma incontrolada.
    - El hecho de que cada objeto sea una “capsula” facilita que sea transportado entre diferentes Sistemas o parte de Sistemas (Reutilización)

© JMA 2001. All rights reserved

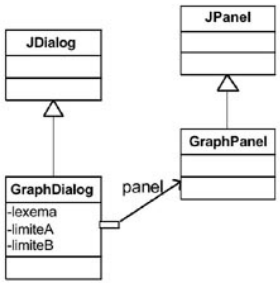
Modificadores de acceso

- Privada:
  - Solo accesible desde los métodos de la propia clase, estrictamente.
- Protegida:
  - Accesible desde los métodos de la propia clase o de uno de sus herederos.
- Publica:
  - Accesible sin restricción desde cualquier método, propio o ajenos
- Interna, compartida, amigos, defecto:
  - Solo accesible desde los métodos de las clases del mismo paquete, ensamblado, módulo, biblioteca, …

© JMA 2001. All rights reserved

Modularidad

- Es la propiedad que permite subdividir una aplicación en partes mas pequeñas (módulos). (*.java)
- Estos módulos deben de ser tan independientes como sea posible.
- Los módulos se pueden compilar por separado, pero normalmente tienen conexiones con otros módulos.



© JMA 2001. All rights reserved

Modularidad

- Siempre que se habla de modularidad, se tiene asociados los conceptos de Dependencia, Acoplamiento y Coherencia.
- Dependencia:
  - Se dicen que el módulo A depende del Módulo B, si al cambiar el modulo B tenemos que cambiar el modulo A.
- Acoplamiento:
  - Define el tipo de dependencia entre módulos
    - FUERTE : Módulos con muchas dependencias
    - DEBIL: Módulos con pocas dependencias
- Coherencia:
  - Determina el tipo de Abstracción conseguida
    - FUERTE: Abstracción Buena
    - DEBIL: Abstracción Mala

© JMA 2001. All rights reserved

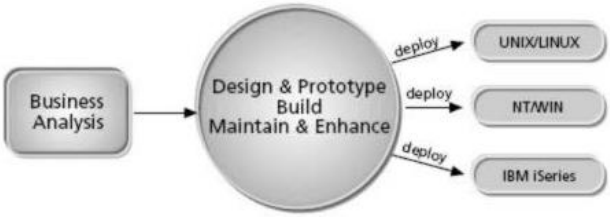
Jerarquía

- Es la propiedad que permite la ordenación de las abstracciones.
- Existen 2 grandes jerarquías en P.O.O
  - Generalización/Especificación (“es una”)
    - Afecta a las estructuras de clases y también es conocido como HERENCIA
    - Relación entre CLASES
  - Agregaciones (“parte de”)
    - Afecta a la estructuras de Objetos
    - Permite el agrupamiento físico de objetos relacionados. (camión, ruedas, motor, ….)

© JMA 2001. All rights reserved

Polimorfismo

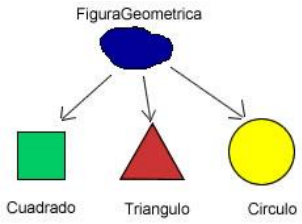
- Es la propiedad que indica la posibilidad de referirse a objetos de clases diferentes mediante el mismo elemento de programa (método), haciendo implementaciones diferentes.
- Por ejemplo:
  - Los mamíferos comen pero no todos de la misma forma.
  - Las piezas del ajedrez se mueven pero no igual.



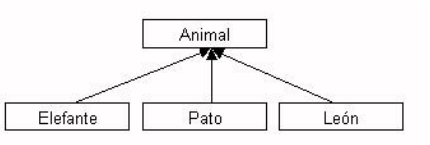
© JMA 2001. All rights reserved

Polimorfismo

- Figuras geométricas.
  - Color
  - Área
  - Perímetro



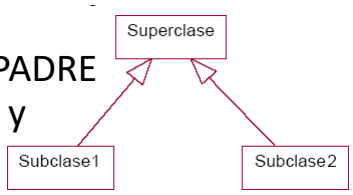
- Animales.
  - Formas de Comer
  - Meverse



© JMA 2001. All rights reserved

Concepto de Herencia

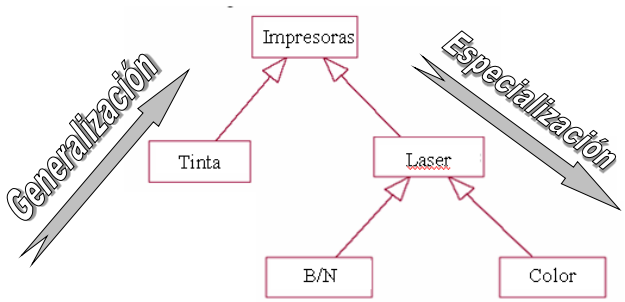
- La Herencia es un método para compartir similitudes entre clases donde los atributos y métodos de varias clases son comunes. Permite la reutilización de la definición de las clases.
- La clase superior es denominada, CLASE BASE, PADRE o SUPERCLASE y de ella heredarán los atributos y métodos.
- La clase inferior es denominado CLASE DERIVADA, HIJA o SUBCLASE y ésta heredará atributos o métodos de la Clase BASE.



© JMA 2001. All rights reserved

Concepto de Herencia

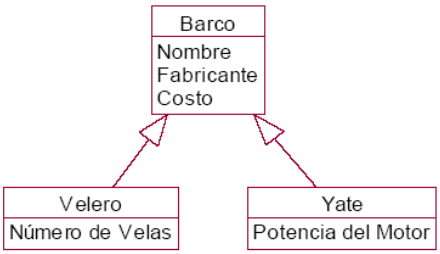
- La Herencia se utiliza para hacer una Jerarquía de clases.
- Según se va bajando en la jerarquía, se va Especializando, mientras que si subimos, vamos Generalizando



© JMA 2001. All rights reserved

Concepto de Herencia

- Cuando creamos un objeto de una clase Derivada, el objeto contiene toda la información de ese clase y también de la clase BASE.



Si creamos un objeto de Yate,
en él dispondremos de los atributos
de BARCO
Nombre, Fabricante, Costo

+ los atributos de Yate

Potencia del Motor

© JMA 2001. All rights reserved

Concepto de Herencia

- CARACTERISTICAS de la HERENCIA
  - Los valores de una instancia derivada incluyen los valores de la/s clase/s base.
  - Cualquier operación de la clase base se puede aplicar a la clase derivada.
  - Las clases derivadas pueden redefinir las clases bases añadiendo atributos o métodos (extensión de la clase).
  - Las clases derivadas no pueden restringir los atributos y métodos heredados.

© JMA 2001. All rights reserved

Sobrescritura de Métodos

- Sobrescritura: Es la posibilidad de que una clase derivada pueda sobrescribir valores de los atributos heredados y la implementación de los métodos.
- La sobrescritura de métodos permite la personalización del heredero respetando el interfaz heredado.
- Gracias a esta sobrescritura de métodos, podemos implementar el concepto de POLIMORFISMO.

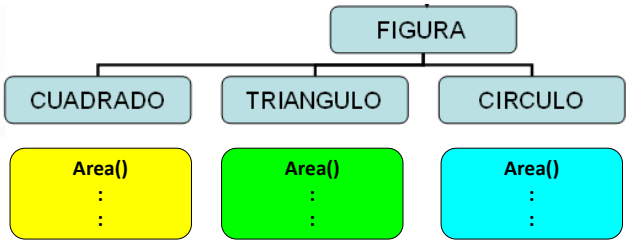
© JMA 2001. All rights reserved

Sobrescritura de Métodos

- Esta sobrescritura puede producirse por varias razones:
  - EXTENSION
    - Una nueva operación es igual a la heredada, excepto que agrega algún comportamiento nuevo.
  - RESTRICCION
    - Restringe el funcionamiento de lo heredado o el tipo de datos heredado para la operación.
  - OPTIMIZACION
    - Para aprovechar atributos y funcionalidades nuevas creadas con ese fin

© JMA 2001. All rights reserved

Sobrescritura de Métodos



Podríamos disponer de un método a nivel de la Clase BASE llamado área y en cada uno de las subclases lo deberíamos sobrescribir para devolver el área correcta del objeto creado

© JMA 2001. All rights reserved

Sobrecarga

- La Firma del método está formada por el número, orden y tipos de los parámetros.
- La sobrecarga permite que una clase tenga varios métodos con el mismo nombre pero con firmas diferentes.
- Los tipos de los argumentos pasados en la invocación del método permite resolver que método se quiere ejecutar.
- El tipo de retorno del método no forma parte de su firma y, por lo tanto, no sirve para diferenciar métodos.
- La sobrecarga permite tener varias versiones del mismo método facilitando su reutilización.

© JMA 2001. All rights reserved

Clases Abstractas

- Hasta ahora todas las clases descritas se conocen como Clases CONCRETAS.
- Cualquiera de estas clases son instanciables, es decir, se pueden crear objetos de ellas en cualquier momento.
- Por el contrario, una CLASE ABSTRACTA, es una clase que no tiene directamente instanciación y por lo tanto no puede tener objetos.
- Las clases Abstractas son un recurso para la HERENCIA.

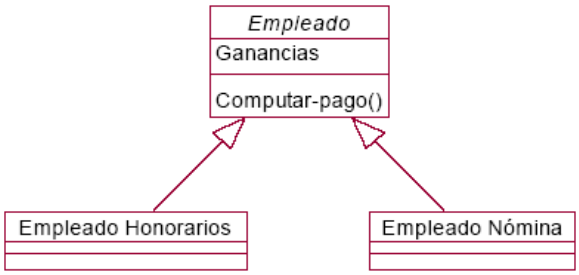
© JMA 2001. All rights reserved

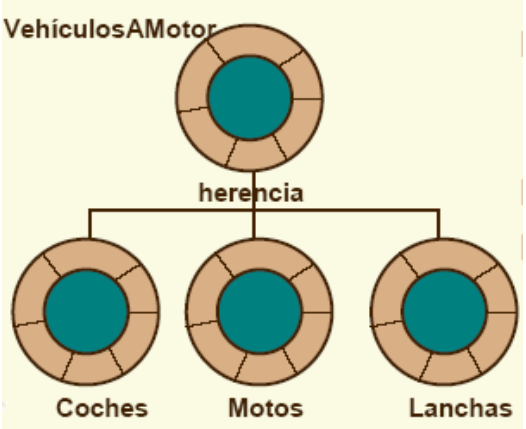
Clases Abstractas

- Las clases Derivadas de ellas, heredarán, como otras cualquiera, y podrán instanciarse.
- Las clases Abstractas pueden:
  - Definir atributos normales
  - Definir e implementar métodos normales
  - Definir métodos ABSTRACTOS
- Para que la clase Derivada sea instanciable, no abstracta, es necesario que en ella se sobrescriban los métodos declarados como Abstractos.

© JMA 2001. All rights reserved

Clases Abstractas





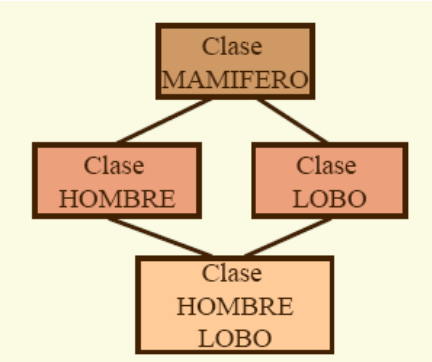
© JMA 2001. All rights reserved

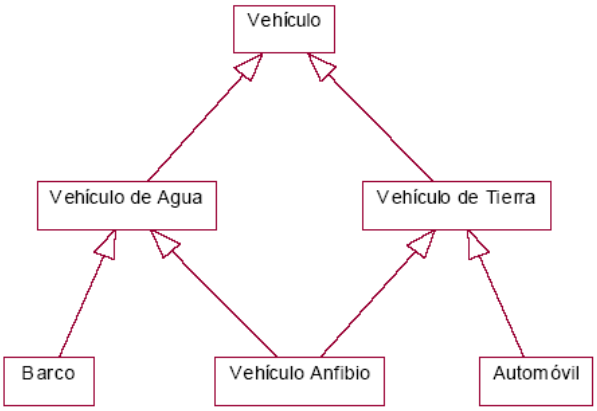
Herencia múltiple

- La Herencia múltiple permite a una clase Derivada o interface tener más de una clase BASE o interface y heredar de ambas.
- Es una característica la mayoría de los lenguajes modernos no implementan en las clases (C++ si, Java no) porque causaba más problemas que los que resolvía (complejidad, conflictos de nombres y ambigüedad).
- Este tipo de herencia puede tener efectos colaterales, por ejemplo cuando se heredan métodos llamados de forma idéntica. Para evitar conflictos, deberíamos de sobrescribirlo.

© JMA 2001. All rights reserved

Herencia múltiple





© JMA 2001. All rights reserved

Herencia múltiple

- La ventaja que proporciona la Herencia Múltiple son:
  - Incorporación e información en una sola operación de Herencia.
  - Oportunidad de reutilizar clases.
- Antes de intentar utilizar esta características debemos estar seguro que es soportada.
  - Lenguajes que la soportan:
    - C++, Centura SQL Windows, CLOS, Eiffel, Object REXX, Perl y Python
  - Lenguajes que NO la soportan:
    - Java, Nemerle, C# y Objective-C

© JMA 2001. All rights reserved

Interfaces

- Un interfaz define explícitamente un grupo de funcionalidades relacionadas, un mecanismo o base de conocimiento común de comunicaciones con los objetos.
- Los interface permiten crear nuevos tipos de datos, pero son tipos abstractos (no instanciables), solo declaran los métodos sin implementar.
- La clase que implementa un interface debe definir todos los métodos declarados en el interface, salvo que sean abstractas en cullo caso pueden delegar la implementación en sus herederos.
- Una clase puede implementar más de un interface.
- Un interface puede reutilizar la definición de uno o varios interfaces: heredar o, mas apropiadamente, sintetizar.

© JMA 2001. All rights reserved

Referencias

- Las referencias son variables o atributos que pueden contener instancias.
- Las referencias se declaran de tipo clase o interfaz.
  - Una referencia de tipo clase puede contener instancias de la clase del tipo o de cualquiera de sus herederos.
  - Una referencia de tipo interfaz puede contener instancias de cualquier clase o heredero que implemente el interfaz.
- La referencia solo conoce los métodos del tipo definido, aunque la instancia tenga funcionalidad adicional no es accesible directamente.
- La instancia de una referencia puede recuperar su tipo original pero hay que indicarlo de forma explicita y requiere comprobación previa.

© JMA 2001. All rights reserved

Vinculación dinámica

- En la orientación a objetos se pueden tener métodos diferentes con el mismo nombre pero no se debe confundir:
  - Sobrecargar: en la misma clase, significa definir nuevos métodos con firmas diferentes.
  - Sobrescribir: implica herencia, significa ocultar un método heredado con una nueva definición del método con la misma firma.
  - Polimorfismo: los métodos con la misma firma pertenecen a clases diferentes.
- La Vinculación dinámica es el mecanismo utilizado para averiguar, en tiempo de ejecución, a qué método debe llamar, a partir del tipo del objeto asignado a una referencia y los argumentos suministrados.

© JMA 2001. All rights reserved

Genéricos

- Los lenguajes orientados a objetos son fuertemente tipados, requieren un tratamiento estricto de tipos para poder resolver la sobrecarga, el polimorfismo, … Esto provoca una inflexibilidad que debe resolverse con herencia, interfaces, …
- Los genéricos introducen el concepto de parámetros de tipo, lo que le permite diseñar clases y métodos que aplazan la especificación de uno o varios tipos hasta que el código de cliente declare y cree una instancia de la clase o el método.
- Los genéricos implementan el concepto de plantillas de clases, igual que una clase permite implementar múltiples instancias, un genérico permite definir múltiples clase.
- Las clases y métodos genéricos combinan reusabilidad, seguridad de tipos y eficacia de una manera en que sus homólogos no genéricos no pueden. Los genéricos se usan frecuentemente con colecciones y los métodos que funcionan en ellas.

© JMA 2001. All rights reserved

Genéricos

```
public class MiLista< T > {
   private List< T > lista;
   private T elementoActual;
   public void add(T item) {
  ...
}

MiLista<String> cadenas;
MiLista<Integer> enteros;
```

© JMA 2001. All rights reserved

Excepciones

- Las excepciones son situaciones anómalas que pueden ocurrir durante la ejecución de las aplicaciones que impiden el correcto funcionamiento de las mismas.
- La mayoría de los lenguajes orientados a objetos gestionan los errores a través del tratamiento de excepciones.
- El tratamiento de las excepciones es obligatorio: cuando se produce una excepción se "rompe el código", el flujo de control salta al primer controlador de excepciones asociado. En caso de no encontrarlo, la excepción se propaga por la pila de llamadas rompiendo el código hasta encontrar el controlador de excepciones apropiado. Si no hay ningún controlador de excepciones para una excepción determinada, el programa deja de ejecutarse con un mensaje de error.
- Trabajar con excepciones implica no solo su tratamiento, también es necesario crearlas y lanzarlas. Las excepciones son instancias de las clases apropiadas que representan la situación anómala. La definición de excepciones propias se hace a través de la herencia de clases.
- En lenguajes como el Java las excepciones son declarativas.

© JMA 2001. All rights reserved

S.O.L.I.D.

- SOLID es el acrónimo que acuñó Michael Feathers, basándose en los 5 principios de la programación orientada a objetos que Robert C. Martin había recopilado en el año 2000 en su articulo “Design Principles and Design Patterns”.
- Los objetivos de estos 5 principios a la hora de escribir código son:
  - Crear un software eficaz: que cumpla con su cometido, sea robusto y estable.
  - Escribir un código limpio y flexible ante los cambios: que se pueda modificar fácilmente según necesidad, que sea reutilizable y mantenible.
  - Permitir la escalabilidad: que acepte ser ampliado con nuevas funcionalidades de manera ágil.
  - Facilitar las pruebas: que permita sustituir las dependencias por dobles de prueba para permitir las pruebas unitarias.
- La aplicación de los principios SOLID está muy relacionada con la comprensión y el uso de patrones de diseño, que permitirán minimizar el acoplamiento (grado de interdependencia que tienen dos unidades de software entre sí) y maximizar la cohesión (grado en que elementos diferentes de un sistema permanecen unidos para alcanzar un mejor resultado que si trabajaran por separado).

© JMA 2001. All rights reserved

S.O.L.I.D.

S	• Single Responsibility Principle (SRP) • Principio de responsabilidad única
O	• Open/Closed Principle (OCP) • Principio de abierto-cerrado
L	• Liskov Substitution Principle (LSP) • Principio de sustitución de Liskov
I	• Interface Segregation Principle (ISP) • Principio de segregación de interfaces
D	• Dependency Inversion Principle (DIP) • Principio de inversión de dependencias

© JMA 2001. All rights reserved

S.O.L.I.D.

- Principio de Responsabilidad Única
  - “A class should have one, and only one, reason to change.”
  - La S del acrónimo del que hablamos hoy se refiere a Single Responsibility Principle (SRP). Según este principio “una clase debería tener una, y solo una, razón para cambiar”. Es esto, precisamente, “razón para cambiar”, lo que Robert C. Martin identifica como “responsabilidad”.
  - El principio de Responsabilidad Única es el más importante y fundamental de SOLID, muy sencillo de explicar, pero el más difícil de seguir en la práctica.
  - El propio Bob resume cómo hacerlo: “Reúne las cosas que cambian por las mismas razones. Separa aquellas que cambian por razones diferentes”.

© JMA 2001. All rights reserved

S.O.L.I.D.

- Principio de Abierto/Cerrado
  - “You should be able to extend a classes behavior, without modifying it.”
  - El segundo principio de SOLID lo formuló Bertrand Meyer en 1988 en su libro “Object Oriented Software Construction” y dice: “Deberías ser capaz de extender el comportamiento de una clase, sin modificarla”. En otras palabras: las clases que usas deberían estar abiertas para poder extenderse y cerradas para modificarse.
  - El principio Open/Closed se suele resolver utilizando polimorfismo.
  - Es importante tener en cuenta el Open/Closed Principle (OCP) a la hora de desarrollar clases, librerías, frameworks, microservicios.

© JMA 2001. All rights reserved

S.O.L.I.D.

- Principio de Sustitución de Liskov
  - “Base classes must be substitutable for their derived classes.”
  - La L de SOLID alude al apellido de quien lo creó, Barbara Liskov, y dice que “las clases base deben poder sustituirse por sus clases derivadas”.
  - Esto significa que los objetos deben poder ser reemplazados por instancias de sus subtipos sin alterar el correcto funcionamiento del sistema o lo que es lo mismo: si en un programa utilizamos cierta clase, deberíamos poder usar cualquiera de sus subclases sin interferir en la funcionalidad del programa.
  - Según Robert C. Martin incumplir el Liskov Substitution Principle (LSP) implica violar también el principio de Abierto/Cerrado.

© JMA 2001. All rights reserved

S.O.L.I.D.

- Principio de Segregación de la Interfaz
  - “Make fine grained interfaces that are client specific.”
  - En el cuarto principio de SOLID, el tío Bob sugiere: “Haz interfaces que sean específicas para un tipo de cliente”, es decir, para una finalidad concreta.
  - En este sentido, según el Interface Segregation Principle (ISP), es preferible contar con muchas interfaces especializadas que definan unos pocos métodos que tener una interface generalista que fuerce a implementar muchos métodos a los que no se dará uso. Los implementadores de Interfaces de clases no deben estar obligados a implementar métodos que no se usan.

© JMA 2001. All rights reserved

S.O.L.I.D.

- Principio de Inversión de Dependencias
  - “Depend on abstractions, not on concretions.”
  - Llegamos al último principio: “Depende de abstracciones, no de clases concretas”.
  - Así, Robert C. Martin recomienda:
    - Los módulos de alto nivel no deberían depender de módulos de bajo nivel. Ambos deberían depender de abstracciones.
    - Las abstracciones no deberían depender de los detalles. Los detalles deberían depender de las abstracciones.
  - El objetivo del Dependency Inversion Principle (DIP) consiste en reducir las dependencias entre los módulos del código, es decir, alcanzar un bajo acoplamiento de las clases. Las abstracciones se logran mediante los interfaces.

© JMA 2001. All rights reserved

La regla KISS

- El principio KISS (del inglés Keep It Simple, Stupid!) «¡Mantenlo sencillo, estúpido!» es un acrónimo usado como principio de diseño.
- El principio KISS establece que la mayoría de sistemas funcionan mejor si se mantienen simples que si se hacen complejos; por ello, la simplicidad debe ser mantenida como un objetivo clave del diseño, y cualquier complejidad innecesaria debe ser evitada.
- Paradójicamente la filosofía KISS no es fácil de lograr porque estamos acostumbrados a creer que cualquier proceso está más trabajado cuanto más complicado parece. Sin embargo, la experiencia nos demuestra que la clave del éxito está en la sencillez y que todo funciona mejor si es simple. ¿Por qué?, porque todo lo que es simple es fácil de cambiar, de recordar, de adaptar y de mantener, y porque la complejidad está ligada al desorden y a las contradicciones lógicas, lo que nos genera incertidumbre y, casi siempre, desinterés y abandono.

© JMA 2001. All rights reserved

La regla DRY

- Las duplicidades son una fuente inagotable de problemas.
- El principio DRY (Don't Repeat Yourself) es una concreción del principio KISS y viene a recordarnos este hecho: No te repitas.
- El código duplicado hace que nuestro código sea más difícil de mantener y comprender además de generar posibles inconsistencias. Hay que evitar el código duplicado siempre. Para ello, dependiendo del caso concreto, la refactorización, la abstracción o el uso de patrones de diseño pueden ser nuestros mejores aliados.
- Lo contrario de DRY es WET (we enjoy typing o "nos lo pasamos bien tecleando"), parece que te pagan por palabras: El código es WET cuando contiene duplicaciones innecesarias.

© JMA 2001. All rights reserved

La regla YAGNI

- El principio YAGNI (You Aren't Gonna Need it o "no lo vas a necesitar") promueve:
  - Mantenga limpia su base de código y no desarrolle código que no sea necesario.
  - Elimine el código si ya no se usa.
- YAGNI tiene su origen en la programación extrema, dónde prima la simplicidad y dónde se opta por no invertir tiempo en dar solución a necesidades futuras todavía inexistentes. La programación extrema se guían por la afirmación "Do The Simplest Thing That Could Possibly Work" (DTSTTCPW): Haz lo más simple que pueda funcionar.
- Las ventajas de este principio:
  - Se ahorra tiempo de desarrollo: no se destina a funcionalidad «no útil».
  - No se pierde tiempo de prueba ni de documentación asociado a la funcionalidad no necesaria.
  - Se evita pensar en las restricciones de la funcionalidad no necesaria.
  - El código no crece más de lo necesario, y no se complica.
- El principio YAGNI no debe afectar a la mantenibilidad ni a la reutilización: se debe realizar una inversión (sobre esfuerzo) siempre que facilite la reutilización (sobrecargas) y la mantenibilidad (arquitectura).

© JMA 2001. All rights reserved

La regla del Boy Scout

- No es suficiente escribir bien el código. El código debe mantenerse limpio con el tiempo. Todos hemos visto que el código se pudre y degrada a medida que pasa el tiempo. Por lo tanto, debemos desempeñar un papel activo en la prevención de esta degradación.
- La regla del Boy Scout dice:
  - «deja el campamento más limpio de como te lo encontraste»
- Ampliándola a otros ámbitos sería algo así como:
  - «deja las cosas mejor de como te las encontraste»
- Sin olvidar:
  - «no hagas lo que no te gusta que te hagan»
- Muchas veces, revisando código, nos encontramos con que el nombre de una variable no es demasiado intuitivo o con un fragmento de código duplicado. Resolviendo este tipo de matices (en vez de mirar hacia otro lado y pasar de largo), estaremos aplicando la regla del Boy Scout.
- Con las pruebas automatizadas adecuadas, es menos probable que la refactorización rompa accidentalmente el código.

© JMA 2001. All rights reserved

Cómo encontrar las clases

- Uno de los sistemas más sencillos de los que dispones para identificar las clases y sus componentes es el análisis gramatical de las especificaciones.
- Según vas leyendo las especificaciones puedes marcar los sustantivos, los verbos y los adjetivos:
  - Los sustantivos indican la existencia de un objeto. Dicho objeto es susceptible de convertirse en una clase. Tienes que tener cuidado con los sinónimos, dado que hacen referencia al mismo objeto, si los representas te encontrarás con objetos duplicados, y la polisemia, con el mismo término hacen referencia diferentes conceptos, en cuyo caso faltaran objetos.
  - Los verbos denotan acciones. Las acciones son los métodos. Por lo tanto, cuando encuentras verbos, estás identificando los métodos de las clases.
  - Los adjetivos y, en muchos casos los adverbios, muestran características propias de los objetos, lo que hemos denominado atributos.
- Mediante este sistema tan simple, puedes realizar la primera aproximación al modelo de datos. En refinamientos sucesivos, eliminas los objetos que no son significativos, quitas los elementos duplicados, incorporas los elementos que faltan, ...

© JMA 2001. All rights reserved

¿Qué es un patrón de diseño?

- Un patrón es un modelo de algo recurrente que sirve de muestra para sacar otros iguales.
- El diseño es una actividad
  - El cómo frente al qué
  - Hacerlo correcto frente a hacer lo correcto
  - Asignar responsabilidades a las clases
- Un patrón describe soluciones comunes a problemas comunes que surgen cuando se desarrolla software dentro de un contexto particular. Son descripciones de soluciones a problemas de diseño general que se repiten una y otra vez.
- Un patrón de diseño nos brinda las siguientes respuestas:
  - Problema: Qué problema de diseño soluciona?
  - Solución: Cómo solucionar el problema?
  - Contexto: Precondiciones para que se pueda aplicar el pattern.
  - Restricciones: Qué afecta la aplicación del pattern

© JMA 2001. All rights reserved

Propiedades no funcionales

- **Reutilización**: nos permite que el código sea reutilizable.
- **Facilidad de modificación**: nos permite que el costo de cambio en el código de alguna funcionalidad sea mínimo.
- **Facilidad de comprensión**: ayuda a que el código sea legible y se entienda mejor su diseño.
- **Facilidad de prueba**: nos permite verificaciones más eficiente y automatizadas.
- **Facilidad de uso**: la aplicación de un patrón es simple, solo basta con adaptar las situaciones plateadas y aplicación de cada patrón.
- **Robustez**: permite que la aplicación sea robusta, bien definida y diseñada.
- **Eficiencia**: permite que el sistema sea eficiente.

© JMA 2001. All rights reserved

¿Por qué usarlos?

- Mejora en la comunicación y documentación
  - “Hay que hacer un Factory Method”
  - Facilita la documentación interna del proyecto.
- Mejora la ingeniería de software.
  - Eleva el nivel del grupo de desarrollo.
- Previene “reinventar la rueda” en diseño
  - Son soluciones ya probadas.
- Mejora la calidad y estructura
  - “¿Cuan grande debe ser una clase?”

© JMA 2001. All rights reserved

¿Qué es un patrón de diseño?

- Ante un problema reiterado ofrece una solución contrastada que lo resuelve.
- Describe el problema en forma sencilla.
- Describe el contexto en que ocurre.
- Describe los pasos a seguir.
- Describe los puntos fuertes y débiles de la solución.
- Describe otros patrones asociados.

© JMA 2001. All rights reserved

Plantillas de patrones

- **Nombre de patrón**: Nombre estándar del patrón por el cual será reconocido en la comunidad.
- **Clasificación del patrón**: creacional, estructural o de comportamiento.
- **Intención**: ¿Qué problema pretende resolver el patrón?
- **Motivación**: Escenario de ejemplo para la aplicación del patrón.
- **Aplicabilidad**: Usos comunes y criterios de aplicabilidad del patrón.
- **Estructura**: Diagramas de clases oportunos para describir las clases que intervienen en el patrón.
- **Participantes**: Enumeración y descripción de las entidades abstractas (y sus roles) que participan en el patrón.

© JMA 2001. All rights reserved

Plantillas de patrones

- **Colaboraciones**: Explicación de las interrelaciones que se dan entre los participantes.
- **Consecuencias**: Consecuencias positivas y negativas en el diseño derivadas de la aplicación del patrón.
- **Implementación**: Técnicas o comentarios oportunos de cara a la implementación del patrón.
- **Código de ejemplo**: Código fuente ejemplo de implementación del patrón.
- **Usos conocidos**: Ejemplos de sistemas reales que usan el patrón.
- **Patrones relacionados**: Referencias cruzadas con otros patrones.

© JMA 2001. All rights reserved

Categorización

- **Patrones de arquitectura o arquitectonicos:**
  - Aquéllos que expresan un esquema organizativo estructural fundamental para sistemas software.
- **Patrones de diseño (GoF):**
  - Aquéllos que expresan esquemas para definir estructuras de diseño (o sus relaciones) con las que construir sistemas software.
- **Patrones de análisis:**
  - Expresan estructuras comunes que surgen durante el análisis.
- **Dialectos:**
  - Patrones de bajo nivel específicos para un lenguaje de programación o entorno concreto.

© JMA 2001. All rights reserved

Categorización

- Los patrones de diseño tuvieron un gran éxito en el mundo de la informática desde principios de la década de 1990 a partir de la publicación del libro Design Patterns escrito por el Gang of Four (GoF), el grupo de los cuatro, compuesto por Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, en el que se recogían 23 patrones de diseño comunes clasificados en 3 categorías:
  - Creación
    - Aislar el proceso de creación de un objeto hacia abstracciones.
  - Estructura
    - Desacopla el sistema, como combinar clases y objetos para dar lugar a estructuras mas complejas.
  - Comportamiento
    - Describe situaciones de control de flujo, las relaciones entre llamadas entre los diferentes objetos y su dimensión temporal.

© JMA 2001. All rights reserved

Patrones creacionales

- **Abstract Factory** (Fábrica abstracta): Permite trabajar con objetos de distintas familias de manera que las familias no se mezclen entre sí y haciendo transparente el tipo de familia concreta que se esté usando.
- **Builder** (Constructor virtual): Abstrae el proceso de creación de un objeto complejo, centralizando dicho proceso en un único punto.
- **Factory Method** (Método de fabricación): Centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la casuística para elegir el subtipo que crear.
- **Prototype** (Prototipo): Crea nuevos objetos clonándolos de una instancia ya existente.
- **Singleton** (Instancia única): Garantiza la existencia de una única instancia para una clase y la creación de un mecanismo de acceso global a dicha instancia.

© JMA 2001. All rights reserved

Patrones estructurales

- **Adapter** (Adaptador): Adapta una interfaz para que pueda ser utilizada por una clase que de otro modo no podría utilizarla.
- **Bridge** (Puente): Desacopla una abstracción de su implementación.
- **Composite** (Objeto compuesto): Permite tratar objetos compuestos como si de uno simple se tratase.
- **Decorator** (Envoltorio): Añade funcionalidad a una clase dinámicamente.
- **Facade** (Fachada): Provee de una interfaz unificada simple para acceder a una interfaz o grupo de interfaces de un subsistema.
- **Flyweight** (Peso ligero): Reduce la redundancia cuando gran cantidad de objetos poseen idéntica información.
- **Proxy**: Mantiene un representante de un objeto.

© JMA 2001. All rights reserved

Patrones de comportamiento

- **Chain of Responsibility** (Cadena de responsabilidad): Permite establecer la línea que deben llevar los mensajes para que los objetos realicen la tarea indicada.
- **Command** (Orden): Encapsula una operación en un objeto, permitiendo ejecutar dicha operación sin necesidad de conocer el contenido de la misma.
- **Interpreter** (Intérprete): Dado un lenguaje, define una gramática para dicho lenguaje, así como las herramientas necesarias para interpretarlo.
- **Iterator** (Iterador): Permite realizar recorridos sobre objetos compuestos independientemente de la implementación de estos.
- **Mediator** (Mediador): Define un objeto que coordine la comunicación entre objetos de distintas clases, pero que funcionan como un conjunto.

© JMA 2001. All rights reserved

Patrones de comportamiento

- **Memento** (Recuerdo): Permite volver a estados anteriores del sistema.
- **Observer** (Observador): Define una dependencia de uno-a-muchos entre objetos, de forma que cuando un objeto cambie de estado se notifique y actualicen automáticamente todos los objetos que dependen de él.
- **State** (Estado): Permite que un objeto modifique su comportamiento cada vez que cambie su estado interno.
- **Strategy** (Estrategia): Permite disponer de varios métodos para resolver un problema y elegir cuál utilizar en tiempo de ejecución.
- **Template Method** (Método plantilla): Define en una operación el esqueleto de un algoritmo, delegando en las subclases algunos de sus pasos, esto permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar su estructura.
- **Visitor** (Visitante): Permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.

© JMA 2001. All rights reserved

Alcance vs Proposito

	Creacionales	Estructurales	Comportamiento
Clases	Factory Method	Adapter	Interpreter Template Method
Instancias	Abstract Factory Builder Protype Singleton	Bridge Composite Decorator Facade Flyweight Proxy	Chain of Responsability Command Iterator Mediator Memento Observer State Strategy Visitor

© JMA 2001. All rights reserved

Fundamentos de diseño

- Programar para las interfaz, no para la herencia.
- Favorecer la composición antes que la herencia.
- Delegación.
- Doble Herencia.
- La herencia de clase define la implementación de una clase a partir de otra (excepto métodos abstractos)
- Un interfaz es un tipo abstracto que define el qué y no el como. Define como se llamara el método o propiedad, pudiendo escribir distinto código en clases no relacionadas.

© JMA 2001. All rights reserved

Herencia

- Ventajas:
  - Implementación ya realizada, reutilización.
  - Útil en situaciones “es un”
- Desventajas:
  - Construir un monstruo.
  - No se puede cambiar la implementación heredada en tiempo de ejecución.
  - No se puede cambiar la jerarquía de clases, solo ampliar.
  - Quebrar la encapsulación.
  - Visibilidad.

© JMA 2001. All rights reserved

Programar con interfaces

- La herencia de clase define la implementación de una clase a partir de otra (excepto métodos abstractos)
- Un interfaz es un tipo abstracto que define como se llamara el método o propiedad, pudiendo escribir distinto código en clases no relacionadas.
- Reutilizar la implementación de la clase base es la mitad de la historia.
- Ventajas:
  - Reducción de dependencias.
  - El cliente desconoce la implementación.
  - La vinculación se realiza en tiempo de ejecución.
  - Da consistencia (contrato).
- Desventaja:
  - Indireccionamiento.

© JMA 2001. All rights reserved

Relaciones entre Clases

- Herencia:
  - estática y eficiente
  - no requiere de un proceso particular, es parte de los mecanismos del lenguaje
  - expone y acopla módulos
  - contradice encapsulamiento
- Composición (asociaciones):
  - dinámica
  - acceso a través de interfaces
  - GoF: “Favorecer composición sobre herencia”
- Dijkstra: “la mayoría de los problemas en ciencias de la computación se resuelven con un nuevo nivel de indireccionamiento”.

© JMA 2001. All rights reserved

Reutilización

- Requiere de un proceso o al menos de una política:
  - Se necesita un repositorio de código
  - Requiere un buen esquema de clasificación para la localización del código mas adecuado
  - Requiere documentación para poderlo usar adecuadamente de forma efectiva
- Diseñar que sea reutilizable es duro
  - Cuales son las abstracciones y objetos mas adecuados
  - Flexibilidad, modularidad, simplicidad, elegancia
  - Tiempo, ensayo y error
  - Los patrones de diseño son facilitadores de la reutilización

© JMA 2001. All rights reserved

Diseñar para los cambios

- Principio Abierto-Cerrado
- Principio No-Clarividencia
- Aspecto clave: estar preparado para los cambios
- Causas de rediseño:
  - Dependencia de la plataforma de hw o sw
  - Dependencia de representación o implementación
  - Crear un objeto especificando su clase explícitamente
  - Dependencia de operaciones concretas (algoritmos)
  - Acoplamiento fuerte
  - Exceso de uso de herencia
  - Inflexibilidad para modificar clases

© JMA 2001. All rights reserved

Causas de rediseño

- Crear un objeto especificando su clase explícitamente.
  - Especificar el nombre de la clase al crear un objeto nos liga a una implementación concreta en vez de a una interfaz. Esto puede complicar los cambios a futuro.
  - Como Evitarlo: Crear los objetos indirectamente
  - Patrones que facilitan su solución: Abstract Factory, Factory Method, Prototype, Inversión de Control e Inyección de Dependencias
- Dependencia de operaciones concretas
  - Cuando especificamos una determinada operación estamos ligando a una forma de satisfacer una petición
  - Como Evitarlo: Evitando ligar las peticiones al código, hacemos mas fácil cambiar el modo de satisfacer una petición, tanto en tiempo de compilación como en tiempo de ejecución
  - Patrones que facilitan su solución: Chain of responsability, Command, Delegate

© JMA 2001. All rights reserved

Causas de rediseño

- Dependencias de plataformas hardware o software
  - Las interfaces externas de los sistemas operativos y las interfaces de programación de aplicaciones (API) varían para las diferentes plataformas hardware y software
  - Como Evitarlo: El software que depende de una plataforma concreta será mas difícil de portar a otras plataformas. Incluso puede resultar difícil mantenerlo actualizado en su plataforma Nativa. Por lo tanto es importante diseñar nuestros sistemas de manera que limiten sus dependencias de plataformas concretas
  - Patrones: Abstract Factory, Bridge
- Dependencias de las representaciones o implementaciones de objetos
  - Los clientes de un objeto que saben como se representa, se almacena, se localiza o implementa, quizá deban ser modificados cuando cambie dicho objeto
  - Como Evitarlo: Ocultar esta información a los clientes previene los cambios rupturistas y en cascada
  - Patrones: Abstract Factory, Bridge, Memento, Proxy

© JMA 2001. All rights reserved

Causas de rediseño

- Dependencias algorítmicas
  - Muchas veces los algoritmos se amplían, optimizan o sustituyen por otros durante el desarrollo y posterior reutilización.
  - Como Evitarlo: Los objetos que dependen de un algoritmo tendrán que cambiar cuando este cambie. Por tanto, aquellos algoritmos que es probable que cambien deberían estar aislados.
  - Patrones: Builder, Iterator, Strategy, Template Method, Visitor
- Fuerte acoplamiento
  - Las clases que están fuertemente acopladas son difíciles de reutilizar por separado, puesto que dependen unas de otras. El fuerte acoplamiento lleva a sistemas Monolíticos, en los que no se puede cambiar o quitar una clase sin entender y cambiar muchas otras. El sistema se convierte en algo muy denso que resulta difícil aprender, portar y mantener.
  - Como Evitarlo: El bajo acoplamiento aumenta la probabilidad de que una clase pueda ser reutilizada ella sola y de que un sistema pueda aprenderse, portarse, modificarse y extenderse mas fácilmente.
  - Patrones: Abstract Factory, Bridge, Chain of responsability, command, Facade, Mediator, Observer

© JMA 2001. All rights reserved

Causas de rediseño

- Añadir funcionalidad mediante la herencia
  - Particularizar un objeto derivando de otra clase suele ser fácil, pero cada nueva clase tiene coste de implementación. Definir una subclase también requiere un profundo conocimiento de la clase base y redefinir una operación puede requerir redefinir otra, o tener que llamar a una operación heredada. Además la herencia puede conducir a una explosión de clases, ya que una simple extensión puede obligar a introducir un montón de clases nuevas.
  - Como Evitarlo: La composición de objetos en general y la delegación en particular proporciona alternativas flexibles a la herencia para combinar comportamiento. Se puede añadir nueva funcionalidad de una aplicación componiendo los objetos existentes de otra forma en vez de definir subclases nuevas de otras clases existentes, pero un uso intensivo de la composición puede hacer que los diseños sean mas difíciles de entender. Muchos patrones de diseño producen estructuras en las que se pueden introducir nueva funcionalidad simplemente definiendo una subclase y componiendo sus instancias con otras existentes
  - Patrones: Bridge, Chain of responsability, Composite, Decorator, Observer, Strategy
- Incapacidad para modificar convenientemente las clases.
  - A veces hay que modificar una clase que no puede ser modificada convenientemente. Quizás necesitemos el código fuente y no lo tengamos (Como puede ser el caso de una biblioteca de clase comercial) o tal vez cualquier cambio requeriría modificar muchas de las subclases existentes
  - Como Evitarlo: Mediante patrones de diseño.
  - Patrones: Adapter, Decorator, Visitor

© JMA 2001. All rights reserved

Antipatrones de Diseño

- Andrew Koenig acuñó por primera vez el término "antipatrón" y la idea esencial era que un antipatrón era algo que parecia una buena idea cuando empiezas, pero que te mete en problemas. Desde entonces, el término se ha utilizado a menudo sólo para indicar cualquier mala idea, pero creo que el enfoque original es más útil.
- Los patrones nos ofrecen una forma de resolver un problema típico, los antipatrones muestran formas de enfrentarse a problemas con consecuencias negativas conocidas. Los antipatrones se basan en la idea de que puede resultar más fácil detectar a priori fallos en el desarrollo del proyecto que elegir el camino correcto o, lo que es lo mismo, descartar las alternativas incorrectas nos puede ayudar a la elección de la mejor alternativa.
- El estudio de los antipatrones es muy útil porque sirve para no escoger malos caminos en el desarrollo de sistemas, teniendo para ello una base documental y así evitar usar simplemente la intuición. Además proporciona una denominación común a problemas que facilita la comunicación entre diferentes desarrolladores.

https://es.wikipedia.org/wiki/Antipatr%C3%B3n_de_dise%C3%B1o

© JMA 2001. All rights reserved

Antipatrones de Diseño

- The Blob (“Clases gigantes”): Este antipatrón se da en objetos con muchos atributos o muchas operaciones. Esto rompe las ventajas de la programación OO, ya que estas clases tan grandes son muy difíciles de mantener, de reusar y de probar. Suele aparecer por un diseño malo o debido a sistemas heredados.
- Lava Flow (“Código muerto”): Código muerto e información de diseño olvidada permanecen congelados en un diseño cambiante. Esto es análogo a un flujo de lava en el que se van endureciendo pedazos de roca.
- Poltergeists (“No se sabe bien lo que hacen algunas clases”): Esta mala práctica se refiere a objetos de un ciclo de vida corto cuya única función suele ser invocar métodos de otros objetos. Esto hace que crezca la dificultad para mantener el sistema.
- Golden Hammer (“Para un martillo todo son clavos”): Este antipatrón se refiere al uso de una tecnología, patrón, arquitectura etc. para cualquier problema o situación incluso cuando es evidente que no va ser útil.
- Copy and paste programming (Programación de copiar y pegar): Programar copiando y modificando código existente en lugar de crear soluciones genéricas.
- Spaghetti Code: se refiere a un código mal estructurado, con la claridad de un plato de spaghettis, no saber donde empieza o acaba.
- Ravioli Code: construir sistemas con multitud de objetos pequeños y muy débilmente conectados.
- Lasagna Code o YAFL (yet another fucking layer, y otra maldita capa más) : Añadir capas innecesarias a un programa, biblioteca o framework. Esta tendencia se extendió bastante después de que se publicase el primer libro sobre patrones.

© JMA 2001. All rights reserved

Antipatrones de Diseño

- Improbability factor ("Factor de improbabilidad"): Asumir que es improbable que un error conocido cause verdaderos problemas.
- Premature optimization ("Optimización prematura"): Realizar optimizaciones sin disponer de la información suficiente para hacerlo con garantías, sacrificando decisiones de diseño.
- Programming by permutation (Programación por permutación): Tratar de aproximarse a una solución modificando el código una y otra vez para ver si acaba por funcionar.
- Stovepipe system (“Aislamiento entre sistemas”): Se refiere a 2 posibles circunstancias: Un sistema que no puede interoperar con el resto de los sistemas de una organización; o pequeños sistemas que están tan unidos entre si que no es posible diferenciar cada uno de ellos, imposibilitando su actualización, refactorización, …
- Vendor Lock-In (“Arquitectura dependiente del producto”): Sucede cuando dependemos de una arquitectura, plataforma o tecnología.
- Architecture by implication (“Arquitectura implícita”): No se especifica la arquitectura del sistema o ignora alguno de sus apartados.
- Design by comittee (“Navaja Suiza”): Se da cuando el proyecto se diseña a través de las reuniones de un comité demasiado numeroso o inexperto.
- Reinvent the wheel ("Reinventar la rueda"): Enfrentarse a las situaciones buscando soluciones desde cero, sin tener en cuenta otras que puedan existir ya para afrontar los mismos problemas.
- Reinventing the square wheel (reinventar la rueda cuadrada): Crear una solución pobre cuando ya existe una buena.

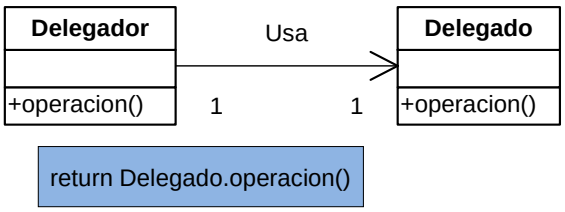
© JMA 2001. All rights reserved

PATRONES FUNDAMENTALES

© JMA 2001. All rights reserved

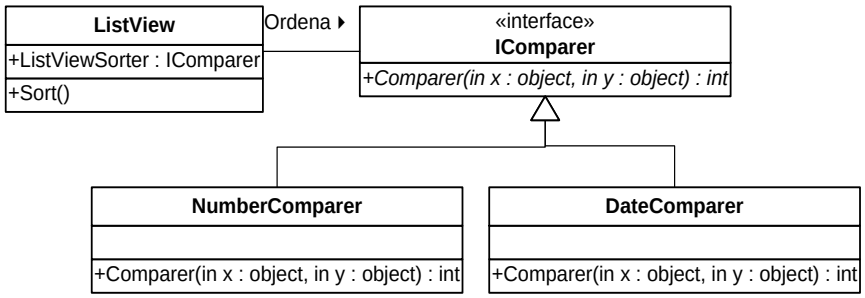
Delegación

- Una forma de componer.
- Un objeto delega un conjunto de operaciones en otro objeto pasado por parámetro.



© JMA 2001. All rights reserved

Delegación



© JMA 2001. All rights reserved

Delegación

- Funciones de orden superior
  - Una función es de orden superior si toma una función como argumento o devuelve una función como resultado.
  - Una función es de orden superior delega los detalles en la función pasada como argumento.
- Las expresiones lambdas (o funciones anónimas o funciones flecha) son una simplificación para pasar funciones como argumentos.

© JMA 2001. All rights reserved

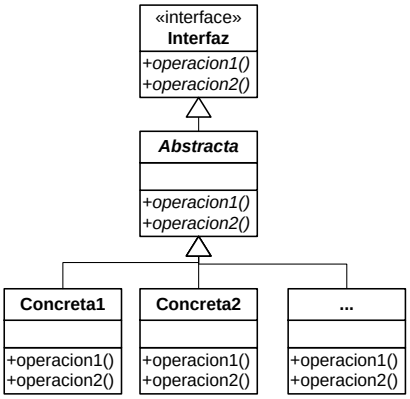
Doble Herencia

- Problema:
  - Organizar clases que tienen un comportamiento parecido para que sean intercambiables y consistentes (polimorfismo).
  - Mantener las clases que implementan el interfaz como internas del proyecto (internal o Friend), pero la interfaz pública.
  - La herencia es rígida, cuando la herencia múltiple no esta soportada, no es posible crear un nuevo heredero de una clase derivada que herede de otra clase base.

© JMA 2001. All rights reserved

Doble Herencia

- Crear un interfaz paralelo a la clase base.
- Crear una clase base abstracta (o no), que implemente dicho interfaz, y sus herederos con implementaciones concretas.
- La clase base puede implementar mas de una interfaz.
- Cualquier clase puede implementar el interfaz independientemente de su jerarquía.
- Todas las referencias (atributos, variables, parámetros, …) se declaran del tipo interfaz.



© JMA 2001. All rights reserved

PATRONES CREACIONALES

© JMA 2001. All rights reserved

Patrones de Creación

- Los patrones creacionales son patrones que abstraen el proceso de instanciación.
- Procuran independizar el sistema de cómo sus objetos son creados, compuestos y representados.
- Encapsulan conocimientos sobre clases concretas usadas por el sistema.
- Dan flexibilidad en qué se crea, quién lo crea, cómo se crea y cuándo se crea.
- Ámbito de objeto
  - Singleton, Factory Method, Prototype, Builder
- Ámbito de clase
  - Abstract Factory

© JMA 2001. All rights reserved

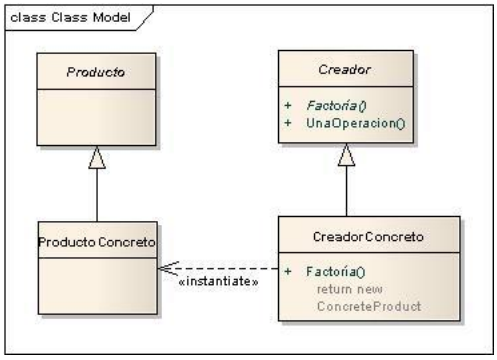
Factory Method

- **Nombre**: También conocido como Método Factoría o Constructor Virtual
- **Clasificación**: Patrón creacional
- **Descripción**:
  - Centraliza en una clase constructora la creación de objetos de un subtipo de un tipo determinado, ocultando al usuario la casuística para elegir el subtipo que crear.
- **Motivación**:
  - Si pensamos en un Framework, rápidamente identificamos que usa clases abstractas para la definición y mantenimiento de las relaciones entre objetos. A menudo es el Framework el responsable de la creación de esos objetos. Pensemos en el ejemplo de un Framework para aplicaciones que pueden presentar multitud de documentación al usuario. Un aplicación no puede prever que tipo de documentación necesita. El Framework tiene que inicializar clases pero solo conoce las clases abstractas.
  - El patrón Factoría encapsula este conocimiento y lo saca fuera del Framework, permitiendo que, mediante nuevas clases, podamos identificar cuál es el documento asociado.

© JMA 2001. All rights reserved

Factory Method

Estructura:



Participantes:

- **Producto**: Define la interfaz de los objetos que la Factoría crea.
- **ProductoConcreto**: Define la interfaz del Producto.
- **Creador**: Declara el método factoría que devuelve un objeto de tipo Producto.
- **CreadorConcreto**: Sobre escribe el método Factoría para que devuelva un ProductoConcreto

© JMA 2001. All rights reserved

Factory Method

- **Colaboraciones**:
  - El Creador busca entre las subclases y devuelve una instancia apropiada de ProductoConcreto.
- **Aplicabilidad**:
  - Una clase de objetos no puede prever la clase de objetos que tiene que crear.
  - Las subclases son las que especifiquen los objetos que se crean.
  - Las clases delegan la responsabilidad en una entre varias clases auxilariares.

© JMA 2001. All rights reserved

Factory Method

- **Consecuencias**:
  - Los métodos Factoría eliminan la necesidad de ligar clases específicas de la aplicación a nuestro código. Dos consecuencias a tener en cuenta son:
    - Proporciona enganches para las subclases.
    - Conexión jerárquica de clases paralelas.
- **Implementación**:
  - Dos variantes principales. El caso en que la clase Creador sea abstracta y no provea de una implementación del método Factoría o el caso en que la clase Creador sea una clase concreta y provea de una implementación del método Factoría
  - Métodos Factoría parametrizados
  - Variantes y cuestiones específicas del lenguaje
  - Uso de plantillas para evitar la herencia
  - Convenciones de nombre: Se suele usar el prefijo Create

© JMA 2001. All rights reserved

Factory Method

- **Usos conocidos** (Complejidad: baja, Popularidad: alta):
  - Muy utilizado en todo tipo de Frameworks
  - JDK. Clase URLConnection, javax.xml.parsers.DocumentBuilderFactory#newDocumentBuilder()
  - Creación de Proxies en middlewares
- **Patrones relacionados:**
  - Factoría Abstracta: Suele implementarse con métodos Factoría
  - Template Method: los métodos fábrica suelen invocarse desde métodos plantilla

© JMA 2001. All rights reserved

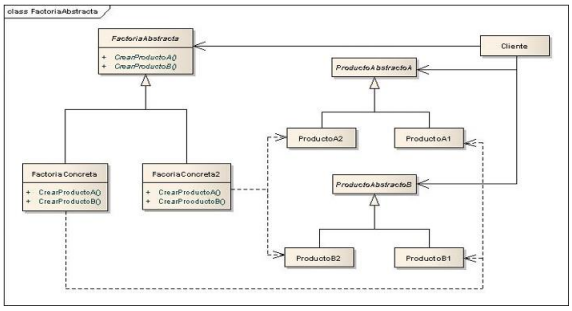
Abstract Factory

- **Nombre**: También es conocido como Factoría Abstracta o Kit
- **Clasificación**: Patrón creacional
- **Descripción**:
  - Es un patrón que define una interfaz para crear familias de objetos relacionados o dependientes sin especificar las clases concretas.
- **Motivación**:
  - Imaginemos la situación de un conjunto de herramientas de interfaces de usuario que soportan varios estándares de representación. En función de cada estándar, el comportamiento y la representación de los elementos de la interfaz varía (scrolls, menu bar, etc..). El cliente no debe cambiar porque cambie la interfaz de usuario. El patrón Factoría Abstracta nos proporciona una solución para esta problemática.

© JMA 2001. All rights reserved

Abstract Factory

Estructura:



Participantes:

- **Cliente**: La clase que llamará a la factoría adecuada ya que necesita crear uno de los objetos que provee la factoría. Es decir, el Cliente lo que quiere es obtener una instancia de alguno de los productos.
- **FactoríaAbstracta**: Es de definición de la interfaces de las factorías. Debe de proveer un método para la obtención de cada objeto que pueda crear.
- **FactoríasConcretas**: Estas son las diferentes familias de productos. Provee de la instancia concreta de la familia que se encarga de crear. De esta forma podemos tener una factoría que cree los elementos gráficos para Windows y otra que los cree para Linux, pudiendo añadir fácilmente (creando una nueva FactoríaConcreta) otra que los cree para MacOS, por ejemplo.
- **ProductoAbstracto**: Definición de las interfaces para la familia de productos genéricos. El Cliente trabajará directamente sobre esta interfaz, que será implementada por los diferentes ProductosConcretos.
- **ProductoConcreto**: Implementación de los diferentes productos.

© JMA 2001. All rights reserved

Abstract Factory

- **Colaboraciones**:
  - Normalmente una instancia de una FactoriaConcreta es creada en tiempo de ejecución. Esta FactoriaConcreta crea el objeto Producto que tiene una implementación particular. Para crear diferentes objetos Producto, los clientes deberán usar diferentes FactoriasConcretas.
- **Aplicabilidad**:
  - Un sistema debe de ser independiente de cómo se crean, componen y representan sus productos.
  - Un sistema debe configurarse con una de entre varias familias de productos.
  - Una familia de productos relacionados están hechos para usarse juntos y se necesita cumplir esa restricción.
  - Se desea ofrecer una biblioteca de clases-producto revelando sus interfaces pero no sus implementaciones.

© JMA 2001. All rights reserved

Abstract Factory

- **Consecuencias**:
  - Aísla al cliente de las clases concretas.
  - Ayuda a controlar la clase de objetos que crea una aplicación.
  - Permite cambiar fácilmente de familia de productos.
  - Promueve la consistencia entre productos, haciendo que una aplicación utilice objetos de una sola familia a la vez
  - La inclusión de nuevos tipos de producto es difícil.

© JMA 2001. All rights reserved

Abstract Factory

- **Implementación**:
  - Factorías como Singletons: Normalmente una aplicación solo necesita una instancia de cada FactoriaConcreta, por lo tanto, lo más indicado es realizarla como Singleton.
  - Creando Productos: FactoríaAbstracta solo declara una interfaz para la creación de productos. Realmente en las clases de ProductoConcreto es donde se crean los mismos. La forma más sencilla es declarar un método factoría por cada Producto.
  - Definiendo factorías extensibles: FactoríaAbstracta define operaciones por cada producto. La aparición de nuevos productos supone una modificación de la interfaz de FactoríaAbstracta.

© JMA 2001. All rights reserved

Abstract Factory

- **Usos conocidos** (Complejidad: media, Popularidad: alta):
  - Es muy común dado que muchos frameworks y bibliotecas lo utilizan para proporcionar una forma de extender y personalizar sus componentes estándar.
  - Cualquier cadena industrial funciona bajo este patrón.
  - java.sql.Connection, javax.xml.datatype.DatatypeFactory, …
  - JSR (Java Specification Requests) e implementaciones de referencia.
- **Patrones relacionados:**
  - Singleton: Las FactoríasConcretas suelen ser a menudo Singleton
  - Factoría: A menudo Clases de FactoriaAbstracta suelen implementarse con métodos de Factoría

© JMA 2001. All rights reserved

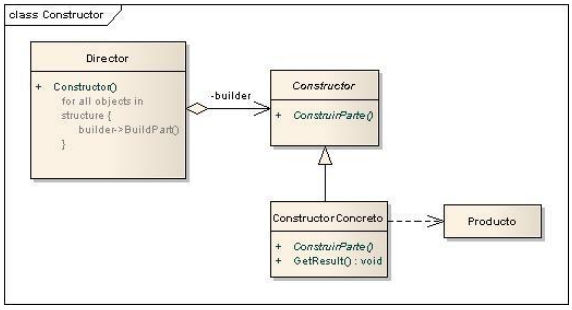
Builder

- **Nombre**: Traducido como Constructor
- **Clasificación**: Patrón creacional
- **Descripción**:
  - Como Patrón de diseño, el patrón Constructor es usado para permitir la creación de una variedad de objetos complejos desde un objeto fuente (Producto). El objeto fuente se compone de una variedad de partes, mas o menos complejas, que contribuyen individualmente a la creación de cada objeto complejo a través de un conjunto de llamadas a interfaces comunes de la clase ConstructorAbstracto.
- **Motivación**:
  - Imaginemos el caso de un lector de textos. A menudo, el lector tendrá que interpretar y convertir un tipo de documentos a otro tipo de lenguaje. El problema radica en que no sabemos muy bien cuantos tipos de lenguaje tendrá que estar preparado para interpretar. Se convierte en necesario crear una estructura que permita añadir una nueva especificación de un lenguaje sin modificar el lector. El patrón Constructor ofrece una solución para ello. Si imaginamos el lector como una clase que realiza un análisis de la información y que se lo pasa a una nueva subclase que realiza la conversión, esta subclase es la que irá añadiendo las especializaciones a nuevos formatos.

© JMA 2001. All rights reserved

Builder

Estructura:



Participantes:

- **Constructor**: Especifica una interfaz abstracta para construir partes del objeto producto.
- **ConstructorConcreto**: Implementa la interfaz de Constructor, construyendo y ensamblando las partes del producto.
- **Director**: Construye un objeto a través de la interfaz Constructor.
- **Producto**: Representa el objeto complejo objetivo de las construcción.

© JMA 2001. All rights reserved

Builder

- **Colaboraciones**:
  - Cliente crea el objeto de Director y configura qué parte necesita del objeto Constructor.
  - Director notifica a Constructor qué parte del producto va a construir.
  - Constructor maneja la petición de Director y añade las partes del producto.
  - El cliente retira el producto del Constructor.
- **Aplicabilidad**:
  - Que el algoritmo para la creación de objetos complejos sea independiente de las partes que construyen el objeto y cómo son ensambladas.
  - Que el proceso de construcción pueda tener diferentes representaciones para el objeto que está construido.

© JMA 2001. All rights reserved

Builder

- **Consecuencias**:
  - Variaciones en la representación interna de un producto: Constructor proporciona una interfaz abstracta para la construcción del producto. La interfaz permite al Constructor ocultar la estructura interna del producto y el proceso de ensamblaje del mismo. Solo es necesario definir un nuevo tipo de constructor para cambiar la representación interna del producto.
  - Se mejora el bajo acoplamiento: El patrón Constructor mejora la modularidad encapsulando el camino para la construcción y la representación del objeto complejo. Cliente no necesita conocer nada acerca de las clases que definen la estructura interna del producto ya que no aparecen en la interfaz del Constructor.

© JMA 2001. All rights reserved

Builder

- **Implementación**:
  - Construcción de la interfaz y ensamblaje: Constructores crean los objetos bajo un modelo paso a paso. La interfaz debe de ser lo suficientemente general para permitir la construcción de todo tipo de objetos de los constructores concretos.
  - ¿Por qué no usar clases abstractas en el producto?: En el caso común, los productos producidos por constructores concretos difieren mucho en su representación, esto es mas difícil de manejar mediante el uso de clases abstractas.
  - Definir métodos vacíos por defecto en el Constructor.

© JMA 2001. All rights reserved

Builder

- **Usos conocidos** (Complejidad: media, Popularidad: alta):
  - Muy utilizado en todo tipo de Frameworks
  - Ampliamente utilizado en la construcción de objetos de configuración
  - Fue un patrón común para el lenguaje SmallTalk
  - Aplicaciones de procesamiento de texto RTF
  - java.lang.StringBuilder, java.time.format.DateTimeFormatterBuilder, …
- **Patrones relacionados**:
  - Factoría abstracta: Ambos están relacionados con la construcción de objetos complejos. La principal diferencia es que Constructor se centra más en la construcción del objeto producto paso a paso, mientras que Factoría Abstracta se centra en la obtención de familias de objetos producto.
  - Composite: A menudo se construye con un Constructor.

© JMA 2001. All rights reserved

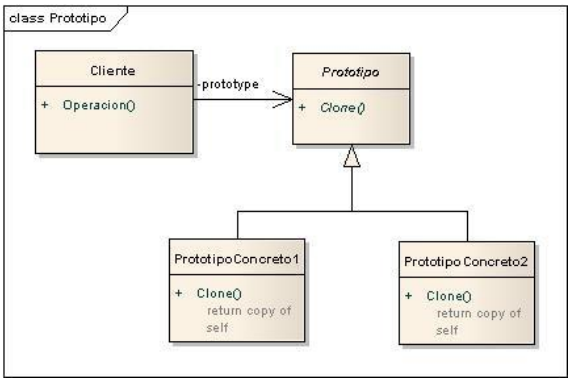
Prototype

- **Nombre**: Traducido como Prototipo
- **Clasificación**: Patrón creacional
- **Descripción**:
  - Este patrón tiene como finalidad crear nuevos objetos duplicándolos, clonando una instancia creada previamente.
- **Motivación**:
  - Este patrón resulta útil en escenarios donde es preciso abstraer la lógica que decide qué tipos de objetos utilizará una aplicación, de la lógica que luego usarán esos objetos en su ejecución. Los motivos de esta separación pueden ser variados, por ejemplo, puede ser que la aplicación deba basarse en alguna configuración o parámetro en tiempo de ejecución para decidir el tipo de objetos que se debe crear. En ese caso, la aplicación necesitará crear nuevos objetos a partir de modelos. Estos modelos, o prototipos, son clonados y el nuevo objeto será una copia exacta de los mismos, con el mismo estado. Esto resulta interesante para crear, en tiempo de ejecución, copias de objetos concretos inicialmente fijados, o costosos de inicializar, también cuando sólo existe un número pequeño de combinaciones diferentes de estado para las instancias de una clase.

© JMA 2001. All rights reserved

Prototype

Estructura:



Participantes:

- **Prototipo**: declara una interfaz para clonarse.
- **PrototipoConcreto**: implementa la operación para clonarse.
- **Cliente**: Crea un nuevo objeto pidiéndole a un Prototipo que se clone.

© JMA 2001. All rights reserved

Prototype

- **Colaboraciones**:
  - Cliente crea un objeto Prototipo para que se clone.
- **Aplicabilidad**:
  - Si el sistema debe de ser independiente de cómo, dónde y cuándo se crean sus productos.
  - Permite especificar instancias en tiempos de ejecución.
  - Se quiere reducir el número de clases.
  - Si las instancias que se generan tienen estados limitados.

© JMA 2001. All rights reserved

Prototype

- **Consecuencias**:
  - Reducción de la herencias.
  - Se puede configurar dinámicamente una aplicación con clases.
  - Especificar nuevos objetos con valores variables.
  - Especificar nuevos objetos con estructuras variables.
  - Reducir las subclases.
  - Habrá tantos prototipos concretos como estados tenga el objeto a instanciar.

© JMA 2001. All rights reserved

Prototype

- **Implementación**:
  - Usar un "Prototipo Manager": Cuando los prototipos se crean y destruyen constantemente manteniendo un registro de los activos, los clientes no quieren ser los responsables del manejo de los prototipos pero pueden almacenar el registro. Un cliente siempre pregunta al registro antes de clonarlo. Este registro de prototipo lo llamamos "Prototipo Manager". El Prototipo Manager es un almacén asociativo que devuelve el prototipo asociado a una clave. De esta manera permitimos al cliente manejar y extender sus prototipos sin necesidad de escribir código.
  - Implementando la operación de clonar: La mayoría de lenguajes da soporte a la clonación, hay que realizar una clonación profunda del objeto.
  - Inicializar los clones.

© JMA 2001. All rights reserved

Prototype

- **Usos conocidos** (Complejidad: baja, Popularidad: media):
  - El patrón Prototype está disponible en Java listo para usarse con una interfaz Cloneable y .NET con ICloneable.
  - Sistema Skethpad donde los usuarios podían formar un objeto compuesto y luego promocionarlo a un prototipo instalándolo en una biblioteca de objetos reutilizables.
- **Patrones relacionados**:
  - Factoría Abstracta: Pueden usarse juntos.
  - Composite: Suele utilizar Prototipo en sus diseños.

© JMA 2001. All rights reserved

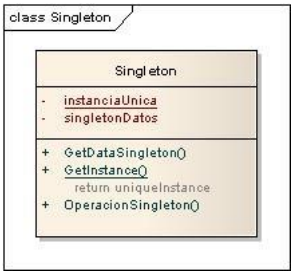
Singleton

- **Nombre**: También es conocido como Singular o Único.
- **Clasificación**: Patrón creacional
- **Descripción**:
  - El patrón de diseño Singleton (instancia única) está diseñado para restringir la creación de objetos pertenecientes a una clase o el valor de un tipo a un único objeto. Su intención consiste en garantizar que una clase sólo tenga una instancia y proporcionar un punto de acceso global a ella. No se encarga de la creación de objetos en sí, sino que se enfoca en la restricción en la creación de un objeto.
- **Motivación**:
  - En algunas ocasiones es muy importante poder garantizar que solo existe una instancia de una clase. Imaginamos la situación de varias impresoras disponibles cuando solo existe un pool que maneja la impresión. Es necesario asegurar que solo existe un objeto dentro del gestor de la impresión. Basándonos en el ejemplo, necesitamos un patrón que nos permita controlar las situaciones que exigen un control de acceso a una instancia bandera, muy habitual en sistemas concurrentes.

© JMA 2001. All rights reserved

Singleton

Estructura:



Participantes:

- Singleton: Define una operación de clase getInstance que permite a los clientes acceder a ella y ademas, es responsable de la creación de la instancia única.

© JMA 2001. All rights reserved

Singleton

- **Colaboraciones**:
  - Los clientes acceden a la instancia única solamente a traves de la operación getInstance.
- **Aplicabilidad**:
  - Este patrón es aplicable en sistemas en los que se desea poder garantizar que solo existe una instancia de una clase.

© JMA 2001. All rights reserved

Singleton

- **Consecuencias**:
  - El uso del patrón Singleton proporciona los siguientes beneficios:
  - Reduce el espacio de nombres. El patrón es una mejora sobre las variables globales. Ya no se reservan nombres para las variables globales, ahora solo existen instancias
  - Controla el acceso a la instancia única, porque la clase Singleton encapsula la única instancia. Así se obtiene control sobre cómo y cuándo se accede a ella.
  - Permite el refinamiento de las operaciones y la representación.
  - Permite un numero variable de instancias. El patrón es fácilmente configurable para permitir más de una instancia
  - Más flexible que las operaciones de clases

© JMA 2001. All rights reserved

Singleton

- **Implementación**:
  - Ocultar el constructor.
  - Declarar como estático el método getInstance.
  - En ambientes concurrentes es necesario usar mecanismos que aseguren la atomicidad del método getInstance.
- **Usos conocidos** (Complejidad: baja, Popularidad: media):
  - Muchos desarrolladores consideran el patrón Singleton un antipatrón, por este motivo, su uso está en declive.
  - Varios patrones pueden implementarse conjuntamente con el patrón Singleton.
    - Factoría
    - Factoría Abstracta
    - Prototipo

© JMA 2001. All rights reserved

Metaclases

- Las clases y las instancias son objetos:
  - ¿Cuál es la clase de una clase?
- Metaclase
  - Clase que describe clases, sus instancias son clases.
  - Propiedades: lista de atributos, lista de variables de clase, lista de métodos, lista de métodos de clase.
- Java, .NET, Ruby y Smalltalk tienen metaclases
- Útil en programación avanzada, cuando se manejan entidades software, por ejemplo en depuradores, inspectores, browsers, …

© JMA 2001. All rights reserved

Metaclases

- Metainformación
- Clases, atributos, métodos, propiedades, etc., son representados por clases.
- Posibilidades
  - Crear o modificar clases en tiempo de ejecución.
  - Parametrizar métodos por clases o métodos.
  - Consultar estructura y comportamiento de una clase.
  - Codificar en tiempo de ejecución
  - …

© JMA 2001. All rights reserved

.NET

REFLEXIÓN

© JMA 2001. All rights reserved

Reflexión

- La clase Assembly representa un ensamblado, que es un bloque de compilación reutilizable, versionable y autodescriptivo.
- Los métodos Load permiten cargar dinámicamente ensamblados.
- Los métodos GetFiles y GetFile permiten acceder a los ficheros almacenados en el área de recursos del ensamblado.
- Los métodos GetTypes y GetType permiten obtener las clases de un ensamblado.
- El método CreateInstance crea una instancia de una clase partiendo de una cadena con el nombre de la clase (instanciación dinámica)

© JMA 2001. All rights reserved

Reflexión

- La clase Type representa la clase de una clase, incluye métodos que retornan información sobre la clase:
  - GetMembers(), GetFields(), GetMethods(), GetConstructors(), GetProperties(), GetEvents(), GetInterfaces(), …
- FieldInfo, MethodInfo y ConstructorInfo, PropertyInfo y EventInfo representan la información sobre atributos, métodos, propiedades y eventos.
- Obtención del tipo:
  - Type t = instancia.GetType();
  - Type t = typeof(clase);

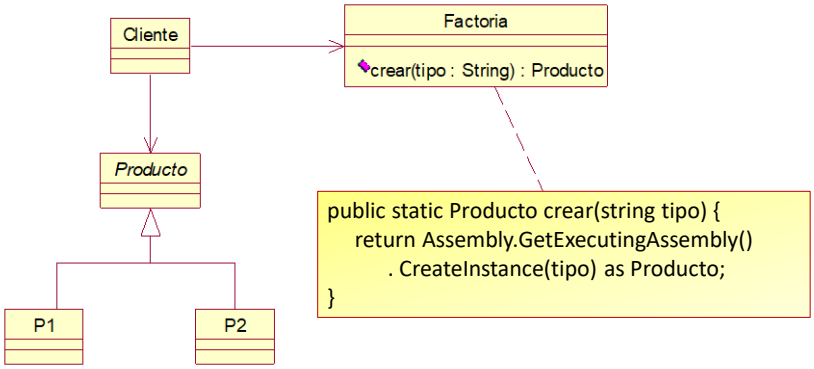
© JMA 2001. All rights reserved

Reflexión

- Se puede utilizar el Type.InvokeMember para invocar un miembro de un tipo.
- Utilizando una cadena con el nombre del miembro y una instancia de la clase representada por Type se puede:
  - Obtener el valor de un atributo o una propiedad de la instancia.
  - Modificar el valor de un atributo o una propiedad de la instancia.
  - Invocar un método de la instancia, pasándole parámetros y obteniendo el valor de retorno.
- También es posible utilizarlo para crear instancias invocando al constructor, pasándole parámetros en caso de ser necesario.

© JMA 2001. All rights reserved

Factoría y Metaclases



© JMA 2001. All rights reserved

Java

REFLEXIÓN

© JMA 2001. All rights reserved

Introducción

- La reflexión se basa en el concepto de que las clases (y el resto de tipos) son objetos y se puede obtener una instancia de la definición de tipo para interactuar con ella.
- La reflexión es comúnmente utilizada por programas que requieren la capacidad de examinar o modificar el comportamiento en tiempo de ejecución de las aplicaciones que se ejecutan en la máquina virtual Java. La reflexión es una técnica muy potente que permite realizar operaciones que de otro modo serían imposibles. La reflexión es una característica relativamente avanzada y solo debe ser utilizada cuando se tenga un conocimiento sólido de los fundamentos del lenguaje.
- La reflexión permite:
  - Codificar en tiempo de ejecución: guardar nombre en cadenas y utilizarlas para cargar dinámicamente clases por su nombre, instanciar las clases cargadas, invocar sus métodos por el nombre, …
  - Explorar las clases para poder enumerar los miembros de las clases, extraer anotaciones, decidir que utilizar, …
  - Poder examinar miembros privados de las clases para crear framewoks avanzados de pruebas, validaciones, serialización, mapeos, …

© JMA 2001. All rights reserved

Inconvenientes

- La reflexión es poderosa, pero no debe usarse de forma indiscriminada. Es preferible evitar su uso si es posible realizar una operación sin usar la reflexión.
- Sobrecarga de rendimiento
  - Debido a que la reflexión involucra tipos que se resuelven dinámicamente, no se pueden realizar ciertas optimizaciones de máquinas virtuales Java. En consecuencia, las operaciones reflexivas tienen un rendimiento más lento que las alternativas no reflexivas y deben evitarse en secciones de código que se llaman con frecuencia en aplicaciones sensibles al rendimiento.
- Restricciones de seguridad
  - Reflection requiere un permiso en tiempo de ejecución que puede no estar presente cuando se ejecuta en contextos de seguridad restringido (Ej. Applet).
- Exposición de internos
  - Dado que la reflexión permite que el código realice operaciones que serían ilegales sin ella, como el acceso a miembros private, el uso de la reflexión puede provocar efectos secundarios inesperados y puede destruir la portabilidad. La reflexión rompe las abstracciones y, por lo tanto, puede cambiar el comportamiento con las actualizaciones de la plataforma.

© JMA 2001. All rights reserved

Objetos de clase

- Las instancias de tipo java.lang.Class encapsulan la definición de los tipos. No se pueden instanciar, es necesario utilizar uno de los siguientes mecanismos:
  - Tipo.class
  - Class.forName(Cadena con el nombre calificado)
  - instancia.getClass()
  - envolvente.TYPE
- Las instancias de Class disponen de métodos que devuelven clases:

  .getSuperclass(): Devuelve la superclase para la clase dada.

  .getClasses(): Devuelve todas las clases, interfaces y enumeraciones públicas que son miembros de la clase, incluidos los miembros heredados.

  .getDeclaredClasses(): Devuelve todas las clases, interfaces y enumeraciones que se declaran explícitamente en la clase incluidos las privados.

© JMA 2001. All rights reserved

Obtención de Miembros

- Están disponibles métodos que devuelven todos los miembros de un tipo o que devuelven uno concreto.
- Así mismo, la versión por defecto solo obtiene los miembros públicos incluidos los heredados, la versión Declared solo obtiene los miembros declarado en la clase incluidos privados y protegidos.
  - getFields(), getDeclaredFields(), getField(), getDeclaredField()
  - getMethods(), getDeclaredMethods(), getMethod(), getDeclaredMethod()
  - getConstructors(), getConstructor()
  - getAnnotations(), getDeclaredAnnotations(), getAnnotation(), getDeclaredAnnotation()

© JMA 2001. All rights reserved

Acceder a Miembros

- Obtener y establecer valores de atributos:

```
Field cmd = instancia.getClass().getDeclaredField("atributo");
cmd.setAccessible(true);
String cad = (String)(cmd.get(instancia));
cmd.set(instancia, "nuevo valor");
```

- Obtener e invocar un método:

```
Method m = instancia.getClass().getDeclaredMethod("metodo");
// m.setAccessible(true);
Object o = m.invoke(instancia);
m.invoke(instancia, "parámetro");
```

- Crear nuevas instancias:

```
Class.newInstance() //solo puede invocar el constructor sin argumentos
Constructor ctor = Console.class.getDeclaredConstructor()[0];
ctor.newInstance()
```

© JMA 2001. All rights reserved

Restringir el accesos a miembros

- La invocación de setAccessible(true) indica que el objeto debe suprimir las comprobaciones del control de acceso del lenguaje Java cuando se utiliza la reflexión. Acceder a miembros privados fuera de la clase puede ser una amenaza para la seguridad de una aplicación Java.
- Se puede restringir usando Security Manager, aunque no se instala de forma predeterminada, por lo que todos los permisos están disponibles. Algunos lanzadores (como el entorno de subprogramas) instalarán un SecurityManager, que a su vez utiliza una política basada en archivos de políticas de forma predeterminada. Se puede cambiar en el archivo "java.security". Otros entornos, como un contenedor de servlets o un contenedor Java EE, también pueden instalar un SecurityManager.
- Se puede habilitar manualmente:

```
System.setSecurityManager(new SecurityManager());
```

- Una vez habilitado el SecurityManager, de forma predeterminada el código ya no puede llamar a "setAccessible".
- La seguridad se define en ${JRE_HOME}/conf/security/java.policy. Para volver a habilitarlo:

```
grant  {
    permission java.lang.reflect.ReflectPermission "suppressAccessChecks";
};
```

© JMA 2001. All rights reserved

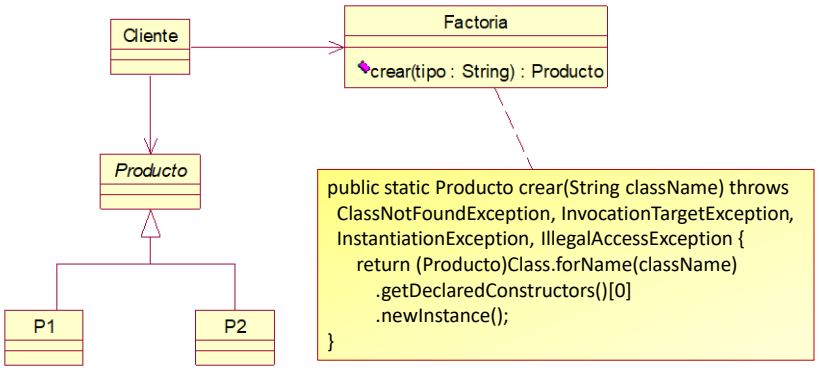
Limitar la accesibilidad de los paquetes

- Los contenedores pueden ocultar el código de implementación agregando a la propiedad de seguridad “package.Access”. Esta propiedad evita que las clases que no son de confianza de otros cargadores de clases se vinculen y utilicen la reflexión sobre la jerarquía de paquetes especificada.

```
private static final String PACKAGE_ACCESS_KEY = "package.access";
static {
   String packageAccess = java.security.Security.getProperty(
      PACKAGE_ACCESS_KEY
   );
   java.security.Security.setProperty(PACKAGE_ACCESS_KEY, (
     (packageAccess == null || packageAccess.trim().isEmpty()) ? "" : (packageAccess + ",")) 
     + "com.example.paquete.protegido"
   );
}
```

© JMA 2001. All rights reserved

Factoría y Metaclases



© JMA 2001. All rights reserved

PATRONES ESTRUCTURALES

© JMA 2001. All rights reserved

Patrones estructurales

- Cómo clases y objetos se combinan para formar estructuras más complejas.
- Patrones basados en herencia
  - Sólo una forma: Adapter
- Patrones basados en composición
  - Describen formas de combinar objetos para obtener nueva funcionalidad
  - Posibilidad de cambiar la composición en tiempo de ejecución.

© JMA 2001. All rights reserved

Adapter

- **Nombre**: También conocido como Adaptador o Wrapper (envoltorio)
- **Clasificación**: Patrón estructural
- **Descripción**:
  - Este patrón convierte la interfaz de una clase en otra interfaz para adaptarla a las necesidades de un desarrollo concreto. El patrón Adaptador permite que clases con interfaces incompatibles puedan trabajar juntas.
- **Motivación**:
  - Es muy frecuente la necesidad de adaptadores para elementos de la vida cotidiana: cargadores de baterías, tipos de enchufe, etc. Si traspasamos esta visión al mundo del software, en algunas ocasiones un conjunto de clases no es reutilizable simplemente por la interfaz que no concuerda con el dominio específico que una aplicación requiere. Es necesario crear un patrón que facilite esta reutilización y que permita no modificar la estructura de códigos del cliente y del servicio.

© JMA 2001. All rights reserved

Adapter

Estructura:

- Una clase adaptadora hereda de la clase o clases que desea adaptar, ofreciendo una interfaz diferente a la clase o clases padre.
- Un objeto adaptador es un objeto que extiende o contiene uno o varios objetos de la clase o clases que se desean adaptar. El interfaz del objeto adaptador será el requerido, e internamente se realizarán las operaciones de conversión necesarias para adaptar las peticiones a la interfaz del objeto y objetos adaptados.

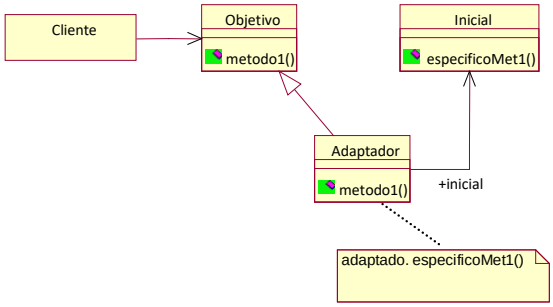
Participantes:

- **Objetivo**: define la interfaz específica del dominio que Cliente usa.
- **Cliente**: colabora con la conformación de objetos para la interfaz Objetivo.
- **Inicial**: define una interfaz o clase existente que necesita adaptarse
- **Adaptador**: adapta la interfaz de Inicial a la interfaz Objetivo

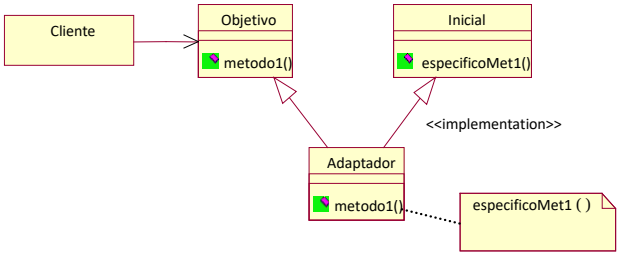
© JMA 2001. All rights reserved

Adapter

Composición



Herencia



© JMA 2001. All rights reserved

Adapter

- **Colaboraciones**:
  - La clase Cliente llama a las operaciones sobre una instancia Adaptador. Posteriormente, el Adaptador llama a las operaciones de Inicial que llevan a cabo el pedido.
- **Aplicabilidad**:
  - Si se quiere reutilizar una clase pero su interfaz no concuerda con nuestras necesidades.
  - Cuando se desea crear una clase reutilizable que coopera con clases no relacionadas, es decir, clases que no tienen necesariamente interfaces compatibles.
  - Cuando se quiera utilizar un componente de caja negra.

© JMA 2001. All rights reserved

Adapter

- **Consecuencias**:
  - El empleo de adaptadores de clase o de objeto tienen diferentes consecuencias.
  - Un adaptador de clase:
    - Adapta Inicial a Objetivo desarrollando una clase Adaptador concreta. Como consecuencia, una clase adaptadora no funcionará cuando se desea adaptar una clase y todas sus subclases.
    - Permite a los Adaptadores sobrescribir algo de comportamiento de Inicial, ya que Adaptador es una subclase de Inicial.
  - Un adaptador de objeto:
    - Permite que un único Adaptador trabaje con muchos interfaces Iniciales, es decir, el Adaptador por sí mismo y las subclases (si es que la tiene). El Adaptador también puede agregar funcionalidad a todos los Iniciales de una sola vez.
    - Hace difícil sobrescribir el comportamiento de Inicial. Esto requerirá derivar Inicial y hacer que Adaptador se refiera a la subclase en lugar que al Inicial por sí mismo.

© JMA 2001. All rights reserved

Adapter

- **Implementación**:
  - Otras cuestiones a considerar cuando se utiliza el patrón Adaptador:
    - ¿Qué nivel de adaptación debe hace el Adaptador? La cantidad de trabajo que el Adaptador debe realizar para adaptar la interfaz Inicial a la inteface Objetivo puede varias. Hay un espectro de trabajo posible que puede ir desde una simple conversión (por ejemplo, cambiando los nombres de las operaciones) hasta el soporte de un conjunto de operaciones enteramente diferentes. La cantidad de trabajo que el Adaptador hace dependerá de la similitud existente entre la interfaz Inicial y la interfaz Objetivo.
    - Adaptadores Pluggables: Una clase es más reutilizable cuando se minimiza la suposición que otras clases deben hacer para utilizarla. Mediante la construcción en una clase de adaptación de una interfaz, se elimina la suposición de que otras clases ven la misma interfaz. Dicho de otra manera, la adaptación de la interfaz nos permite incorporar nuestra clase en sistemas existentes que pueden esperar diferentes interfaces para la clase.
  - Para realizar una implementación correcta del patrón se recomienda:
    - Crear una nueva clase que será el Adaptador, que extienda del componente existente e implemente la interfaz obligatoria. De este modo tenemos la funcionalidad que queríamos y cumplimos la condición de implementar la interfaz

© JMA 2001. All rights reserved

Adapter

- **Usos conocidos** (Complejidad baja:, Popularidad: alta):
  - Un ejemplo dentro del JDK:
    - Para gestionar eventos un objeto debe de implementar EventListener
    - Para gestionar eventos de tipo Window debe de implementar la interfaz WindowListener que extiende deEventListener
    - WindowListener proporciona siete métodos pero en muchas ocasiones no se usan mas de tres, ¿es posible evitar la definición de clases derivadas de WindowListener que no añaden comportamiento para la mayoría de los métodos? JDK proporciona la clase WindowAdapter para tal fin
  - Se utiliza muy a menudo en sistemas basados en algún código heredado (legacy).
- **Patrones relacionados**:
  - El patrón de diseño Proxy define un representante para un objeto pero no cambia su interfaz.
  - El patrón de diseño Decorador amplia la funcionalidad de un objeto sin modificar su interfaz.

© JMA 2001. All rights reserved

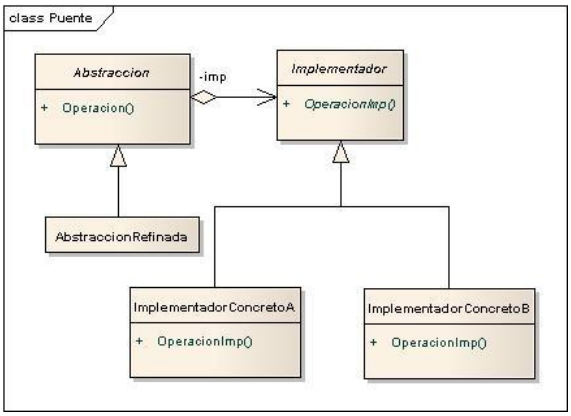
Bridge

- **Nombre**: También conocido como Puente o Handle/Body
- **Clasificación**: Patrón estructural
- **Descripción**:
  - La idea de este patrón es desacoplar una abstracción de su implementación, de manera que ambas puedan ser modificadas independientemente sin necesidad de que la alteración de una afecte a la otra.
- **Motivación**:
  - Cuando una abstracción puede tener varias posibles implementaciones, normalmente hacemos uso de la herencia para acomodar esta necesidad. Una clase abstracta define de la interfaz de la abstracción y las subclases concretas lo implementan. Esto puede no es los suficientemente flexible, es difícil de mantener y modificar y no permite reutilizar los componentes. Necesitamos un patrón que solucione esta problemática.

© JMA 2001. All rights reserved

Bridge

Estructura:



Participantes:

- **Abstracción**: Define una interface abstracta. Mantiene una referencia (puente) a un objeto de tipo Implementador.
- **AbstracciónRefinada**: Extiende la interface definida por Abstracción
- **Implementador**: define la interface para la implementación de clases. Esta interface no tiene que corresponder exactamente con la interface de Abstracción; de hecho, las dos interfaces pueden ser bastante diferentes. Típicamente, la interface Implementador provee sólo operaciones primitivas, y Abstracción define operaciones de alto nivel basadas en estas primitivas.
- **ImplementadorConcreto**: Implementa la interface de Implementador y define su implementación concreta.

© JMA 2001. All rights reserved

Bridge

- **Colaboraciones**:
  - Abstraccion emite los pedidos de los clientes a su objeto Implementador.
- **Aplicabilidad**:
  - Se desea evitar un enlace permanente entre la abstracción y su implementación. Esto puede ser debido a que la implementación debe ser seleccionada o cambiada en tiempo de ejecución.
  - Tanto las abstracciones como sus implementaciones deben ser extensibles por medio de subclases. En este caso, el patrón permite combinar abstracciones e implementaciones diferentes y extenderlas independientemente.
  - Cambios en la implementación de una abstracción no deben impactar en los clientes, es decir, su código no debe tener que ser recompilado.
  - Se desea esconder la implementación de una abstracción completamente a los clientes.
  - Se desea compartir una implementación entre múltiples objetos (quizá usando contadores) y este hecho debe ser escondido a los clientes.

© JMA 2001. All rights reserved

Bridge

- **Consecuencias**:
  - Desacopla interface e implementación: Una implementación no es limitada permanentemente a una interface. La implementación de una abstracción puede ser configurada en tiempo de ejecución. Además, le es posible a un objeto cambiar su implementación en tiempo de ejecución. Desacoplar Abstracción e Implementador también elimina las dependencias sobre la implementación en tiempo de compilación. Cambiar una clase de implementación no requiere recompilar la clase Abstracción y sus clientes. Esta propiedad es esencial cuando te debes asegurar la compatibilidad binaria entre diferentes versiones de una biblioteca de clases. Es más, este desacoplamiento fomenta las capas, lo que pueden conducir a un sistema mejor estructurado. La parte de alto nivel de un sistema sólo tiene que conocer Abstracción e Implementador.
  - Mejora la extensibilidad: Se puede extender las jerarquías de Abstracción e Implementador independientemente.
  - Esconde los detalles de la implementación a los clientes.

© JMA 2001. All rights reserved

Bridge

- **Implementación**:
  - Sólo un Implementador: En situaciones donde existe sólo una implementación, crear una clase Implementador abstracta no es necesario. Esto es un caso especial del patrón; hay una relación uno-a-uno entre Abstracción e Implementador. Sin embargo, esta separación es aún muy útil cuando un cambio en la implementación de una clase no debe afectar a sus clientes existentes, es decir, ellos no deben ser recompilados, sólo relinkeados.
  - Creando el objeto Implementador adecuado: ¿Cómo, cuándo, dónde y qué clase Implementor instanciar cuando hay más de una? Si Abstraccion conoce todas las clases ImplementadorConcreto y puede instanciar una de ellas en su constructor; puede decidir cuál instanciar dependiendo de los parámetros del constructor, se puede elegir una implementación inicial por defecto y cambiarla después acorde al uso o es posible delegar la decisión a otro objeto en conjunto.
  - En C++:
    - Compartiendo Implementadores: El estilo Handle/Body en C++ puede ser usado para compartir implementaciones de muchos objetos. Body almacena una cuenta de referencia que la clase Handle incrementa y decrementa.
    - Usando herencia múltiple: Se puede usar herencia múltiple para asociar una interfaz con su implementación. Creamos una clase Abstracción padre que sea abstracta, además de Abstracciones concretas mediante clases que heredan de ella. Por otro lado se tienen las clases que implementan la funcionalidad con una estructura similar: una clase Implementador abstracta padre, y todas las clases hijas necesarias que implementan la funcionalidad de todas las maneras necesarias. La relación se da entre la clase abstracta Abstracción y la clase abstracta Implementador, delegando la primera la implementación en la segunda, que a su vez la delega en las implementaciones concretas.

© JMA 2001. All rights reserved

Bridge

- **Usos conocidos** (Complejidad: alta, Popularidad: baja):
  - Es de especial utilidad a la hora de tratar con aplicaciones multiplataforma, soportar varios tipos de servidores de bases de datos, o trabajar con varios proveedores de API de un cierto tipo (por ejemplo, plataformas en la nube, redes sociales, etc.).
  - El DOM de KDE3 esta basado en un bridge
  - ET++ , framework para aplicaciones
- **Patrones relacionados**:
  - Factoría Abstracta permite crear y configurar un Puente particular
  - El patrón Adaptador tiene también como objetivo hacer trabajar juntas clases con distinta interfaz, pero en general se aplica a sistemas que existen. El patrón Puente se suele aplicar al empezar un diseño, para permitir que las Abstracciones e Implementadores evolucionen independientemente
  - Bridge, State, Strategy (y, hasta cierto punto, Adapter) tienen estructuras muy similares. De hecho, todos estos patrones se basan en la composición, que consiste en delegar trabajo a otros objetos. Sin embargo, todos ellos solucionan problemas diferentes.

© JMA 2001. All rights reserved

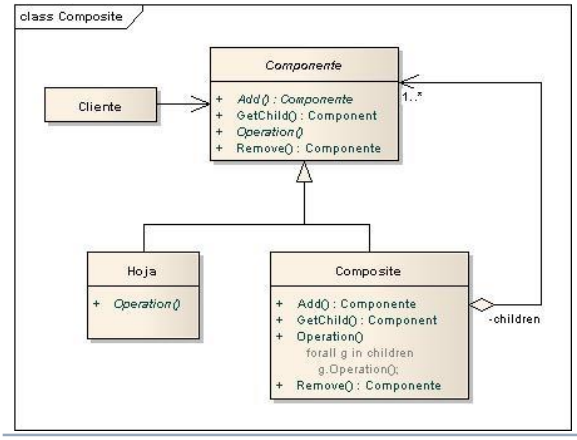
Composite

- **Nombre**: Traducido como Composición
- **Clasificación**: Patrón estructural
- **Descripción**:
  - El patrón Composite sirve para construir objetos complejos a partir de otros más simples y similares entre sí, gracias a la composición recursiva y a una estructura en forma de árbol. Esto simplifica el tratamiento de los objetos creados, ya que, al poseer todos ellos una interfaz común, se tratan todos de la misma manera.
- **Motivación**:
  - Pensamos en una aplicación gráfica de componentes sencillos (lineas, texto, etc) Imaginemos que necesitamos crear una serie de clases para guardar información acerca de una figuras geométricas. Se crearán las clases Círculo, Cuadrado y Triángulo, que heredarán todas de la clase Figura y que implementarán el método pintar(). Además, se necesita poder tratar también grupos de imágenes, ya que nuestro programa permite seleccionar varias de estas figuras a la vez para moverlas por la pantalla. Para la implementación de estos grupos de Figuras podríamos caer en la tentación de crear una clase particular separada de las anteriores llamada GrupoDeImágenes, también con un método pintar().
  - Podríamos pensar la idea de separar en clases privadas componentes (figuras) y contenedores (grupos) pero tiene el problema de que, para cada uno de los dos tipo de objeto, el método pintar() tendrá una implementación diferente, aumentando la complejidad del sistema. Lo lógico es crear una clase abstracta que represente componentes y contenedores de la cual todas heredan, y que define sus operaciones. Por ejemplo, en este caso, se podría crear un clase Gráfico de la que hereden tanto la clase Figura como la clase GrupoDeImagenes, y que incluya el método pintar(). Además, ésta última tendría una relación todo-parte de multiplicidad con Gráfico: un GrupoDeImágenes contendría varios Gráficos, ya fuesen éstos Cuadrados, Triángulos, u otras clases GrupoDeImágenes
  - De esta forma, el patrón Composite da una solución elegante a este problema, de la que además resulta en una implementación más sencilla

© JMA 2001. All rights reserved

Composite

Estructura:



Participantes:

- **Componente**: Define la interfaz común para los objetos de la composición. Además, define la interfaz para acceder y gestionar los hijos. Implementa un comportamiento por defecto común a las subclases
- **Hoja**: Representa los objetos sin hijos de la composición. Define el comportamiento de los mismos.
- **Composite**: Define el comportamiento para los componentes que tienen hijos. Almacena los componentes con hijos e implementa las operaciones para la gestión de hijos
- **Cliente**: Manipula los objetos de la composición a través de la interfaz

© JMA 2001. All rights reserved

Composite

- **Colaboraciones**:
  - Cliente usa el interfaz de Componente para interactuar con objetos en la estructura de Composite.
  - Si el receptor es una Hoja, entonces la petición es manejada directamente.
  - Si el receptor es un Composite, lanza la petición a sus hijos.
- **Aplicabilidad**:
  - Se quiere realizar jerarquías de objetos del tipo "todo-parte"
  - Se quiere ser capaz de ignorar la diferencia entre los objetos individuales y composiciones de objetos. Los clientes tratarán a todos los objetos de la estructura compuesta uniformemente.

© JMA 2001. All rights reserved

Composite

- **Consecuencias**:
  - Define jerarquías de clases hechas de objetos primitivos y compuestos. Si el código cliente espera un objeto simple, puede recibir también uno compuesto.
  - Simplifica el cliente, ya que los objetos simples y compuestos se tratan homogéneamente.
  - Facilita la incorporación de nuevos tipos de componentes.
  - Puede hacer el diseño demasiado general.

© JMA 2001. All rights reserved

Composite

- **Implementación**:
  - Referencias explicitas a los padres: Con ello se simplifica algunas operaciones de la estructura compuesta. Lo mejor es definirlas en la clase Componente. Gestionarlas al añadir/eliminar elementos de un Composite.
  - Compartir Componentes: Es muy útil por el ahorro de memoria que supone. La gestión del componente con varios padres puede llegar a ser compleja
  - Maximizar la interfaz del componente: Dar un comportamiento por defecto que sobrescribirán las subclases.
  - Orden de los hijos: En ocasiones los hijos presentan un orden y hay que tenerlo en cuenta para la implementación
  - Declaración de las operaciones de manejo de hijos: Definirlas en la raíz Componente y darle una implementación por defecto permite aumentar la transparencia, pero se pierde seguridad (¿Cómo evitar que añada/elimine objetos a una hoja?). Si las definimos en la clase Composite se obtiene más seguridad pero se pierde transparencia al no existir una interfaz uniforme.

© JMA 2001. All rights reserved

Composite

- **Usos conocidos** (Complejidad: media, Popularidad: media):
  - Se utiliza a menudo para representar jerarquías de componentes de interfaz de usuario o el código que trabaja con gráficos.
  - Las clases java.awt.Component (Componente), java.awt.Container (Contenedor), java.awt.Panel (Contenedor concreto), java.awt.Button (Boton).
  - Documentos HTML, XML, …
- **Patrones relacionados**:
  - Se puede utilizar un Builder al crear árboles Composite complejos.
  - Chain of Responsibility se utiliza a menudo junto a Composite.
  - Se puede utilizar el patrón Visitor para ejecutar una operación sobre un árbol Composite entero.

© JMA 2001. All rights reserved

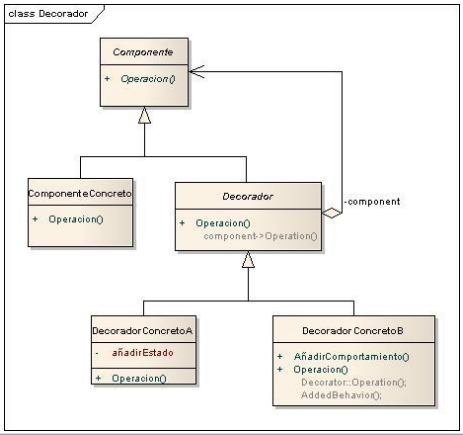
Decorator

- **Nombre**: También es conocido como Decorador o Wrapper
- **Clasificación**: Patrón estructural
- **Descripción**:
  - El patrón responde a la necesidad de añadir dinámicamente funcionalidad a un objeto. Esto nos permite no tener que crear sucesivas clases que hereden de la primera incorporando la nueva funcionalidad, sino otras que la implementan y se asocian a la primera.
- **Motivación**:
  - A veces se quiere añadir funcionalidad a un objeto concreto, no a una clase entera. Supongamos, por ejemplo, un kit de herramientas de una interfaz gráfica para GUIs que proporciona soporte para marcos, barras de desplazamiento, etc. Podríamos intentar resolver esta situación mediante el empleo de la herencia entre clases, pero no es posible ya que este mecanismo no es flexible y la funcionalidad se añade estáticamente.
  - Si definimos una clase que envuelva al componente y proporcione la funcionalidad deseada, tendremos un diseño más flexible y transparente. El patrón Decorador nos proporciona esta solución.

© JMA 2001. All rights reserved

Decorator

Estructura:



Participantes:

- **Componente**: Define la interfaz de los objetos a los que se puede añadir responsabilidades de manera dinámica
- **ComponenteConcreto**: Define un objeto al que añadir responsabilidades de manera dinámica
- **Decorador**: Mantiene una referencia al objeto componente y define una interfaz conforme a la del componente
- **DecoradorConcreto**: Añade responsabilidades al componente al que referencia

© JMA 2001. All rights reserved

Decorator

- **Colaboraciones**:
  - Decorador adelanta peticiones al objeto Componente. En ocasiones se realizan operaciones previas y posteriores al lanzamiento de la petición
- **Aplicabilidad**:
  - Añadir responsabilidades a objetos concretos de manera dinámica y transparente, esto es, sin afectar a otros objetos.
  - Para el tratamiento de responsabilidades que se pueden otorgar o derogar.
  - Cuando la herencia sea muy compleja porque implique la creación de múltiples subclases para dar completitud a todas las combinaciones posibles.

© JMA 2001. All rights reserved

Decorator

- **Consecuencias**:
  - Es la más flexible que la herencia estática. Las responsabilidades se añade y eliminan dinámicamente. Facilita definir una propiedad varias veces.
  - Evita una complejidad en las clases más altas en las jerarquías. Se facilita la definición de nuevos decoradores
  - Un decorador y el componente al que se refieren no son idénticos (tienen identificador distinto).
  - Provoca la creación de muchos objetos pequeños parecidos y encadenados, dificultando la depuración.

- **Implementación**:
  - Un componente y su decorador deben compartir la misma interfaz.
  - Se puede omitir la clase abstracta Decorador si solo se va a definir una responsabilidad.
  - Mantener una clase Componente ligera. En caso contrario podemos ir heredando características en las subclases que no necesitan.
  - La diferencia con el patrón estrategia es que en este patrón el componente no cambia.

© JMA 2001. All rights reserved

Decorator

- **Usos conocidos** (Complejidad: media, Popularidad: media):
  - Es bastante común, especialmente en el código relacionado con los flujos (streams).
  - Se emplea con frecuencia para dar funcionalidades a los Toolkits.
  - En .NET, los métodos extensores cubren la misma funcionalidad.
- **Patrones relacionados**:
  - Estrategia: Decorador modifica la piel del objeto, mientras que Estrategia modifica el funcionamiento interno.
  - Adaptador: Se diferencian en que Adaptador modifica la interfaz del objeto mientas que el decorador no lo realiza.

© JMA 2001. All rights reserved

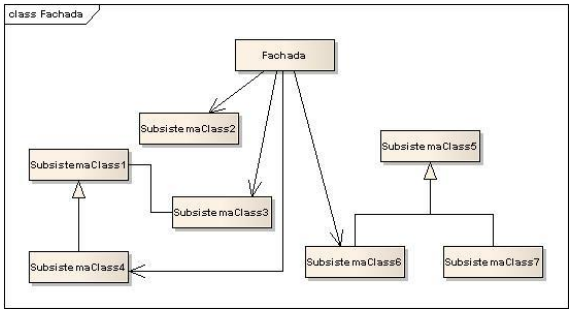
Facade

- **Nombre**: Traducido como Fachada
- **Clasificación**: Patrón estructural
- **Descripción**:
  - El patrón de diseño Fachada sirve para proveer de una interfaz unificada sencilla que haga de intermediaria entre un cliente y una interfaz o grupo de interfaces más complejas.
- **Motivación**:
  - Estructurar un sistema complejo en pequeños subsistemas reduce la complejidad global. Un objetivo común del diseño es reducir y minimizar las comunicaciones entre subsistemas. Una forma de conseguirlo puede ser introduciendo un objeto Fachada que provea una interfaz simple ampliando la facilidad del sistema.
  - Si pensamos, por ejemplo, en la estructura de un entorno de programación, podemos asumir que el compilador tendrá subclases encargadas de realizar cada uno de los distintos aspectos de la compilación, como Analex, AnaSin, TabSim, ASA, etc. Estas clases serán comúnmente accedidas por el Editor, el Enlazador, el Depurador, etc. Por lo tanto, surge la necesidad de crear una fachada para unificar estas llamadas.

© JMA 2001. All rights reserved

Facade

Estructura:



Participantes:

- **Fachada**: Delegan la petición del cliente en objetos de los subsistemas
- **Clases de Subsistemas**: Implementan las funcionalidades concretas del subsistema.

© JMA 2001. All rights reserved

Facade

- **Colaboraciones**:
  - Los clientes se comunican con el subsistema a través de la Fachada, que reenvía las peticiones a los objetos del subsistema adecuado.
  - Los clientes que usan fachada no necesitan conocer ni acceder directamente a los objetos.
- **Aplicabilidad**:
  - Se quiere proporcionar una interfaz sencilla para un subsistema complejo.
  - Se quiere desacoplar un subsistema de sus clientes o de otros subsistemas convirtiéndolo en más independiente y portable.
  - Se quiere dividir el sistema por niveles, actuando las fachadas como entradas a cada nivel.

© JMA 2001. All rights reserved

Facade

- **Consecuencias**:
  - Oculta a los clientes el comportamiento del subsistema, haciendo que este sea mas fácil de usar.
  - Promueve un débil acoplamiento entre subsistemas
- **Implementación**:
  - Se puede reducir aun mas el acoplamiento haciendo que la Fachada sea una clase abstracta, de forma que se pueda escoger entre diferentes implementaciones del subsistema.
  - Hay lenguajes que facilita la definición de ámbitos internos (o de paquete) para las clases privadas a un subsistema.

© JMA 2001. All rights reserved

Facade

- **Usos conocidos** (Complejidad: baja, Popularidad: media):
  - En Java las clases java.awt.Graphics y java.awt.Font son ejemplos de uso de este patrón.
- **Patrones relacionados**:
  - Las Fachadas suelen ser Singletons.
  - Se puede anteponer una Fachada a un conjunto de Adaptadores.

© JMA 2001. All rights reserved

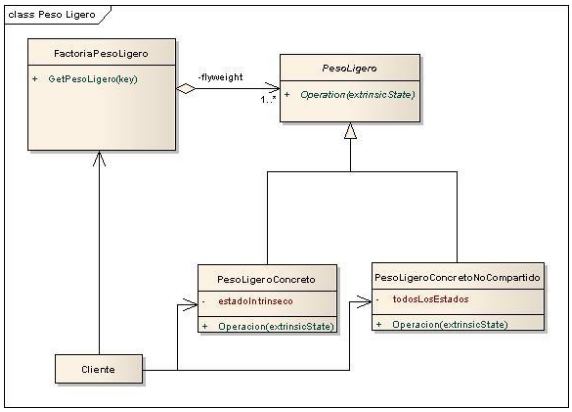
Flyweight

- **Nombre**: También conocido como Peso Mosca o Peso Ligero
- **Clasificación**: Patrón estructural
- **Descripción**:
  - Este patrón sirve para eliminar o reducir la redundancia cuando tenemos gran cantidad de objetos que contienen información idéntica, además de lograr un equilibrio entre flexibilidad y rendimiento (uso de recursos).
- **Motivación**:
  - Describe como almacenar un gran número de objetos sin un gran coste. Para conseguir esto se utilizan objetos que almacenan los estados inmutables compartidos y que pueden ser utilizados por varios objetos simultáneamente.

© JMA 2001. All rights reserved

Flyweight

Estructura:



Participantes:

- **PesoLigero**: Declaran una interfaz a través de la cual los PesosLigeros pueden recibir y actuar sobre estados no compartidos.
- **PesoLigeroConcreto**: Cualquier estado que almacene debe de ser independiente de contexto. Deben de ser compartibles. Implemente la interfaz de PesoLigero.
- **PesoLigeroConcretoNoCompartido**: No todas las subclases de PesoLigero tiene que ser compartidas.
- **Cliente**: Contiene las referencias a los PesosLigeros.
- **FactoriaPesoLigero**: Crea y gestiona los objetos PesoLigero, garantiza que los objetos PesoLigero se comparten de forma adecuada.

© JMA 2001. All rights reserved

Flyweight

- **Colaboraciones**:
  - Los clientes no crean directamente los objetos de PesoLigero, sino que invocan a la FactoriaPesoLigero. El estado es mantenido por el cliente y pasado cuando se invocan métodos que lo requieren.
- **Aplicabilidad**:
  - Se utiliza un gran número de objetos.
  - El coste del almacenamiento es alto debido a la cantidad de objetos.
  - La mayor parte del estado de los objetos no cambia y pueden ser creados como comunes.
  - Muchos objetos pueden ser reemplazados por unos pocos una vez que han sido borrados los estados comunes.
  - La mayor parte del estado del objeto puede ser extrínseco.

© JMA 2001. All rights reserved

Flyweight

- **Consecuencias**:
  - Puede introducir costes run-time debido a la necesidad de calcular y transferir el estado extrínseco.
  - La ganancia en espacio depende de varios factores:
    - la reducción en el número de instancias
    - el tamaño del estado intrínseco por objeto
    - si el estado extrínseco es calculado o almacenado
  - Interesa un estado extrínseco calculado
  - Se suele combinar con el Composite.

© JMA 2001. All rights reserved

Flyweight

- **Implementación**:
  - Asegurar que el rendimiento en los objetos es un tema primordial, y que el cliente está dispuesto a asumir el reajuste.
  - Dividir el objetivo principal en estados:
    - Estado Intrínseco (elementos que se puedan compartir o son comunes)
    - Estado Extrínseco (elementos particulares a cada tipo).
  - Retirar los elementos con Estado Extrínseco de los atributos de la clase y añadir una llamada a métodos
  - Crear una fábrica que pueda almacenar y reutilizar las instancias existentes de clases.
  - El cliente debe usar la fábrica en vez de utilizar el operador new si requiere de creación de objetos.
  - El cliente (o un tercero) debe revisar los Estados Extrínsecos y reemplazar esos estados con métodos de la clase

© JMA 2001. All rights reserved

Flyweight

- **Usos conocidos** (Complejidad: alta, Popularidad: baja):
  - Se usa mucho en las interfaces de usuario de las aplicaciones.
- **Patrones relacionados**:
  - Se puede implementar nodos de hoja compartidos del árbol Composite como Flyweights para ahorrar memoria RAM.

© JMA 2001. All rights reserved

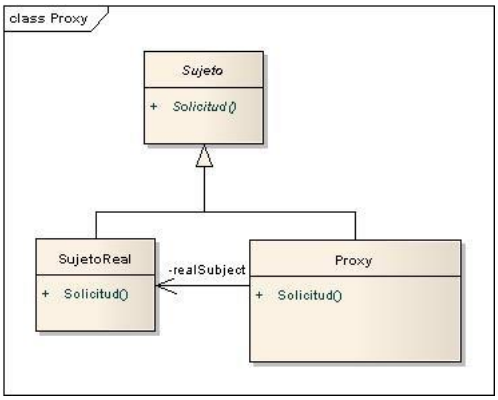
Proxy

- **Nombre**: También conocido como Apoderado o Procurador
- **Clasificación**: Patrón estructural
- **Descripción**:
  - Este patrón nos proporciona un sustituto o representante de otro objeto para controlar el acceso a este.
- **Motivación**:
  - El principal motivo de no permitir el acceso directo a un objeto es reducir el coste que supone la creación y mantenimiento del mismos hasta que realmente es necesario. Imaginemos un documento con muchas imágenes embebidas, supone un consumo de recursos pero a su vez es necesaria la rapidez a la hora de abrir el documento. Si partimos de la idea de que no todas las imágenes se ven en el documento a la vez, solo se irán cargando aquellas que sean necesarias. Esta idea se puede implementar usando este patrón.

© JMA 2001. All rights reserved

Proxy

Estructura:



Participantes:

- **Sujeto**: Define la interfaz común para Proxy y SujetoReal, de manera que pueda usarse un Proxy donde se espera un SujetoReal.
- **SujetoReal**: define el objeto real representado.
- **Proxy**: Mantiene un referencia que permite al proxy acceder al objeto real. Proporciona una interfaz idéntica a la del Sujeto de manera que pueda ser sustituido por un SujetoReal. Controla el acceso al SujetoReal, puede ser el responsable de su creación y borrado.

© JMA 2001. All rights reserved

Proxy

- **Colaboraciones**:
  - Proxy "suplanta" al SujetoReal cuando se considera apropiado.
- **Aplicabilidad**:
  - Útil cuando se desea retrasar la instanciación de un objeto hasta que sea necesario usarlo.
  - Proporciona un representante local de un objeto situado en otro espacio de direcciones.
  - Uso en sistemas concurrentes mediante cerrojo, controlando el acceso al objeto original.

© JMA 2001. All rights reserved

Proxy

- **Consecuencias**:
  - Un Proxy puede ocultar el hecho de que un objeto reside en un espacio de direcciones diferente.
  - Puede llevar a cabo optimizaciones tales como crear un objeto por encargo.
  - Permiten realizar tareas de mantenimiento adicionales cuando se accede a un objeto.
- **Implementación**:
  - Proxy no siempre conoce el tipo de objeto del SujetoReal. En el caso de que no le sea necesario instanciar objetos del SujetoReal no se necesita saber su tipo de objeto, pero en el caso habitual sí.
  - Sobrecarga de operadores de acceso.

© JMA 2001. All rights reserved

Proxy

- **Usos conocidos** (Complejidad: media, Popularidad: baja):
  - Carga diferida o perezosa (lazy) en los ORM
  - De uso común en los fameworks de comunicaciones
- **Patrones relacionados**:
  - Facade es similar a Proxy en el sentido de que ambos pueden almacenar temporalmente una entidad compleja e inicializarla por su cuenta. Al contrario que Facade, Proxy tiene la misma interfaz que su objeto de servicio, lo que hace que sean intercambiables.
  - Con Adapter se accede a un objeto existente a través de una interfaz diferente. Con Proxy, la interfaz sigue siendo la misma. Con Decorator se accede al objeto a través de una interfaz mejorada pero propósitos muy distintos.

© JMA 2001. All rights reserved

PATRONES DE COMPORTAMIENTO

© JMA 2001. All rights reserved

Patrones de comportamiento

- Relacionados con la asignación de responsabilidades entre clases.
- Enfatizan la colaboración entre objetos.
- Caracterizan un flujo de control más o menos complejo que será transparente al que utilice el patrón.
- Basados en herencia:
  - Template Method e Interpreter
- Basados en composición:
  - Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Visitor

© JMA 2001. All rights reserved

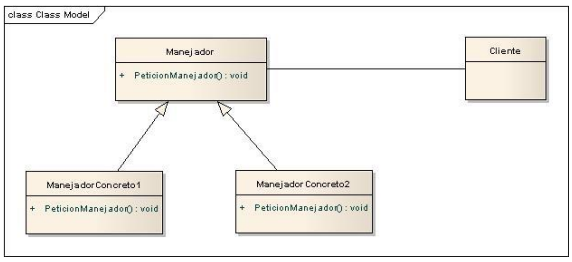
Chain of Responsibility

- **Nombre**: Traducido como Cadena de responsabilidades
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - El patrón de diseño Cadena de responsabilidades permite establecer una cadena de objetos receptores a través de los cuales se pasa una petición formulada por un objeto emisor. Cualquiera de los objetos receptores puede responder a la petición en función de un criterio establecido.
- **Motivación**:
  - Imaginemos un contexto gráfico donde se puede obtener información de ayuda de cada elemento haciendo click sobre él. La información dependerá de la parte de la interfaz donde se pinche. Puede darse la situación que dos "botones" iguales difieran en la información a mostrar. Para este tipo de problemas necesitamos un patrón que permita manejar dónde se produce un evento, quién es el responsable del mismo y cuál es la respuesta adecuada al mismo.

© JMA 2001. All rights reserved

Chain of Responsibility

Estructura:



Participantes:

- **Cliente**: Inicia la petición que llega a la cadena en busca del responsable.
- **Manejador**: Define una interfaz para manejar peticiones.
- **ManejadorConcreto**: Define las responsabilidades de cada componente. Si puede manejar una petición, la procesa, en caso contrario busca al siguiente.

© JMA 2001. All rights reserved

Chain of Responsibility

- **Colaboraciones**:
  - Cuando un Cliente realiza una petición, está se propaga por el Manejador hasta que un ManejadorConcreto asume la responsabilidad de procesarla.
- **Aplicabilidad**:
  - Más de un objeto necesita manejar una respuesta y el manejador no es conocido a priori.
  - Se quiere poder realizar una petición sin conocer a quién hay que solicitarlo.
  - El conjunto de objetos que pueden procesar una respuesta pueden ser especificados de forma dinámica.

© JMA 2001. All rights reserved

Chain of Responsibility

- **Consecuencias**:
  - Añade flexibilidad en la asignación de responsabilidades de los objetos: Se puede variar la asignación de responsabilidades añadiendo nuevos manejadores o modificándolos.
  - La recepción no esta asegurada: Si la cadena no esta bien configurada puede no cubrir todas las peticiones.
  - Reduce el acoplamiento: El patrón permite que un objeto envíe una petición y sepa que va a ser tratada correctamente, pero tanto el receptor como el emisor no conocen nada el uno del otro.
- **Implementación**:
  - Implementar la cadena sucesora: Hay dos posibilidades de implementación. La primera es definir nuevos enlaces, ya sea en la clase Manejador o en las clase ManejadorConcreto. La segunda posibilidad es emplear enlaces ya existentes, por ejemplo empleando el patrón Composición.
  - Conexión de los sucesores: Los propios ManejadoresConcretos son los encargados de propagar incondicionalmente la petición. Las referencias deben de estar definidas.
  - Representación de las peticiones: Se puede emplear el paso por parámetros o variables mediante una función manejadora, o hacer uso de invocaciones a operaciones insertadas en el código.

© JMA 2001. All rights reserved

Chain of Responsibility

- **Usos conocidos** (Complejidad: media, Popularidad: baja):
  - Manejadores de eventos, para la propagación de eventos a los componentes padre (bubbling) de las clases GUI.
  - Interceptores y Filtros.
  - Editores gráficos.
  - Protocolos industriales a nivel bajo.
- **Patrones relacionados**:
  - Chain of Responsibility se utiliza a menudo junto a Composite.
  - Chain of Responsibility, Command, Mediator y Observer abordan distintas formas de conectar emisores y receptores de solicitudes

© JMA 2001. All rights reserved

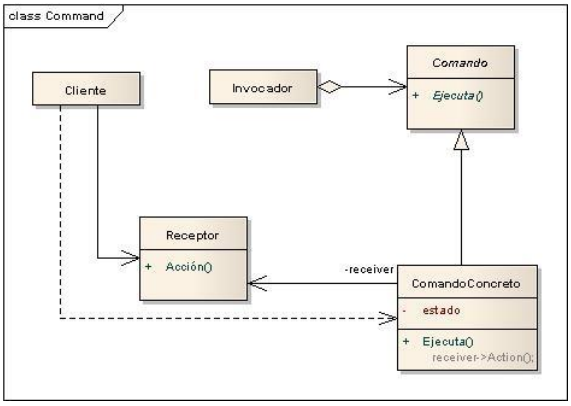
Command

- **Nombre**: Este patrón también es conocido como Comando, Acción o Transacción
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Este patrón permite solicitar una operación a un objeto sin conocer realmente el contenido de esta operación, ni el receptor real de la misma. Para ello se encapsula la petición como un objeto, con lo que además se facilita la parametrización de los métodos.
- **Motivación**:
  - El concepto de comando es muy ambiguo y complejo pero esta muy extendido. Podemos mencionar como ejemplos:
    - Interpretes de comandos del sistema operativo
    - Lenguajes de macros
    - Gestores de bases de datos
    - Protocoles de servidores de internet.
  - Es necesario encontrar un patrón que permita parametrizar a los clientes con distintas peticiones, hacer colas de peticiones, llevar un registro de las peticiones realizadas y que pueda deshacer el estado y el efecto de las mismas. El patrón Comando nos permite realizar todo esto.

© JMA 2001. All rights reserved

Command

Estructura:



Participantes:

- **Comando**: Clase que ofrece un interfaz para la ejecución de órdenes. Define los métodos do y undo que se implementarán en cada clase concreta.
- **ComandoConcreto**: Clase que implementa una orden concreta y sus métodos do y undo. Su constructor debe inicializar los parámetros de la orden.
- **Cliente**: crea un comando concreto e indica a quién va dirigido
- **Invocador**: contiene el comando asociado a la petición
- **Receptor**: Sabe realizar las operaciones asociadas a una peticion. Cualquier clase puede actuar como receptor.

© JMA 2001. All rights reserved

Command

- **Colaboraciones**:
  - Cliente crea un objeto ComandoConcreto y asocia un receptor.
  - Invocador almacena el objeto ComandoConcreto.
  - Invocador lanza la petición llamando al método do de Comando. Si es un Comando con operaciones de vuelta atrás, ComandoConcreto almacena el estado previo a la modificación.
  - El objeto ComandoConcreto invoca a operaciones en el receptor.
- **Aplicabilidad**:
  - Parametrizar objetos mediante una acción
  - Especificar, encolar y ejecutar peticiones en distintos momentos (el objeto Comando tiene un tiempo de vida distinto al de la petición)
  - Independizar el momento de petición del de ejecución.
  - Mantener operaciones que permitan deshacer la petición.
  - Acciones que permitan la recuperación del sistema.
  - Interfaz común que permita invocar las acciones de modo uniforme y extender el sistema de modo sencillo.

© JMA 2001. All rights reserved

Command

- **Consecuencias**:
  - Desacopla el objeto que invoca a la operación del que sabe como llevarla a cabo.
  - Los comandos son entidades de "primer orden", se pueden manipular y extender como cualquier otro objeto.
  - Se pueden crear macro-comandos mediante el uso del patrón Compositor.
  - Es fácil añadir nuevos comandos ya que no es necesario modificar las clases existentes.
- **Implementación**:
  - A la hora de hacer recomendaciones sobre la implementación de este patrón es necesario tener claro los dos conceptos siguientes:
    - ¿Cómo de inteligente debe ser un Comando? Un Comando puede implementarse con dos habilidades consideradas extremas. La primera de estas habilidades es invocar una acción en el receptor. La segunda de estas habilidades consideradas extremas es implementar el comportamiento sin delegar (muy útil para comandos independientes del receptor o cuando este está implícito)
    - ¿Cómo implementar las operaciones de vuelta atrás y rehacer (operaciones undo y redo)? Hay que tener en cuenta que deben de ser operaciones adicionales al patrón. Es necesario almacenar el estado previo a la ejecución del Comando: objeto receptor, argumentos de la operación, valores del receptor que puedan ser modificados. Si se crea una tabla de estados anteriores se permitirá volver hacia atrás todos los estados.

© JMA 2001. All rights reserved

Command

- **Usos conocidos** (Complejidad: baja, Popularidad: alta):
  - Es muy común, la mayoría de las veces se utiliza como alternativa a las retrollamadas (callbacks) para parametrizar elementos UI con acciones. También se utiliza para poner tareas en cola, realizar el seguimiento del historial de operaciones, etc.
- **Patrones relacionados**:
  - Factoría: Ofrece una forma alternativa de llamar a los comandos además del uso del command manager.
  - Intérprete: Se puede implementar un pequeño Intérprete mediante clases Comando.
  - Plantilla: Puede servir para implementar la lógica de Deshacer de forma un tanto automática o a alto nivel.
  - Composite: Permite realizar agrupaciones de comandos de forma similar a una macro.
  - Prototipo: Hay quien lo utiliza para implementar la copia del comando al histórico de comandos.

© JMA 2001. All rights reserved

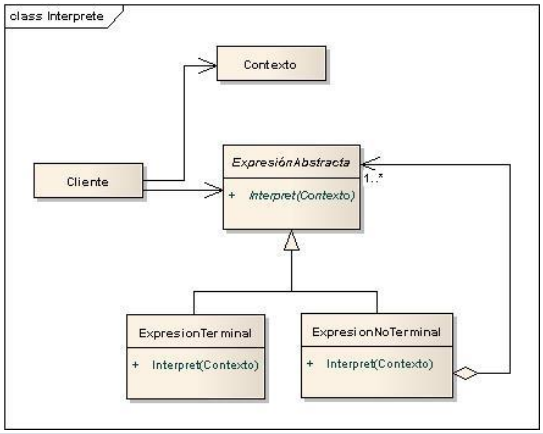
Interpreter

- **Nombre**: Traducido como Intérprete
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - El Intérprete es un patrón de diseño que, dado un lenguaje, define una representación para su gramática junto con un intérprete del lenguaje. Se usa para definir un lenguaje para construir expresiones regulares que representen cadenas a buscar dentro de otras cadenas. Además, en general, para definir un lenguaje que permita representar las distintas instancias de una familia de problemas.
- **Motivación**:
  - Existen problemas particulares que pueden expresarse en función de algún lenguaje. Es necesario construir un intérprete de dicho lenguaje. El patrón define reglas gramaticales del lenguaje, así como la realización y comprensión de sentencias del mismo.

© JMA 2001. All rights reserved

Interpreter

Estructura:



Participantes:

- **ExpresiónAbstracta**: Declara una operación abstracta Intérprete que es común a todos los nodos del árbol de sintaxis abstracta.
- **ExpresiónTerminal**: Una instancia es requerida por cada aparición en un sentencia. implementa un operación Intérprete asociada a cada símbolo terminal.
- **ExpresiónNoTerminal**: Para cada regla es necesario un tipo de clase.
- **Cliente**: Construye el árbol sintáctico de ExpresionesNoTerminales, e instancias de la clase ExpresiónTerminal.
- **Contexto**: Contiene información global para el interpretador.

© JMA 2001. All rights reserved

Interpreter

- **Colaboraciones**:
  - Cliente construye una sentencia compuesta de ExpresionesNoTerminales y ExpresionesTerminales. Luego inicializa el Contexto e invoca al interpretador
  - Cada nodo correspondiente a ExpresionesNoTerminales define al interpretador en función de subexpresiones
  - Las ExpresionesTerminales evalúan las expresiones anteriores
  - Las operaciones en cada nodo utilizan el Contexto para almacenar y acceder al estado del interpretador
- **Aplicabilidad**:
  - Cuando tratamos con gramáticas simples, en caso contrario la mejor opción es utilizar parsers.
  - La eficiencia no es uno de los aspectos más importantes. Hay que traducir la entrada a una forma inmediata.

© JMA 2001. All rights reserved

Interpreter

- **Consecuencias**:
  - Es fácil cambiar y extender la gramática.
  - Implementar la gramática es sencillo.
  - Las gramáticas complejas son difíciles de mantener.
  - Se puede añadir nuevas interpretaciones de los símbolos.
- **Implementación**:
  - La creación del árbol sintáctico no se especifica: Se puede usar un parser o realizarlo en cliente.
  - Definiendo la operación de interprete: No hace falta implementarla en cada clase de expresiones. Es recomendable el uso del patrón Visitante.
  - Compartiendo símbolos terminales mediante el patrón Peso Ligero.

© JMA 2001. All rights reserved

Interpreter

- **Usos conocidos** (Complejidad: alta, Popularidad: baja):
  - Se usa en compiladores con lenguajes orientados a objetos como SmallTalk.
- **Patrones relacionados**:
  - Composite: El árbol sintáctico es un Composite.
  - Peso Ligero: Permite compartir símbolos terminales del árbol sintáctico.
  - Visitor: Puede usarse para mantener el comportamiento de cada nodo del árbol de sintaxis abstracta en una sola clase.

© JMA 2001. All rights reserved

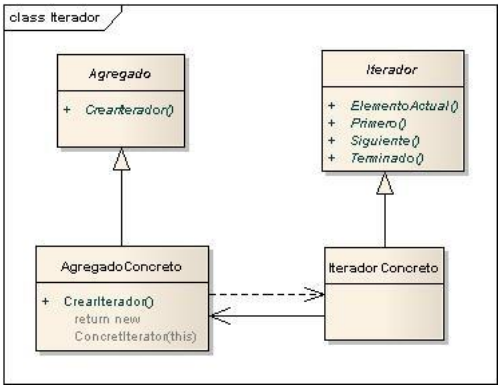
Iterator

- **Nombre**: También conocido como Iterador o Cursor.
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Es un patrón define una interfaz que declara los métodos necesarios para acceder secuencialmente a un grupo de objetos de una colección.
- **Motivación**:
  - Debemos de disponer de un medio para navegar por los datos de una lista sin exponer su estructura interna. Así mismo, se debe recorrer la lista de varias maneras pero sin que esto signifique añadir operaciones a la lista por cada tipo de recorrido. En algunos casos nos interesa mantener varios recorridos simultáneamente.
  - El patrón Iterador permite hacer todo esto. La solución que propone es dar la responsabilidad de recorre la lista a un objeto Iterador. Este define una interfaz para acceder a los elementos de la lista.

© JMA 2001. All rights reserved

Iterator

Estructura:



Participantes:

- **Agregado**: define la interfaz para crear un objeto Iterador.
- **Iterador**: define la interfaz para agregar y recorrer elementos.
- **AgregadoConcreto**: Implementa la interfaz para crear un objeto Iterador.
- **IteradorConcreto**: Implementa la interfaz de Iterador y mantiene la posición actual del recorrido.

© JMA 2001. All rights reserved

Iterator

- **Colaboraciones**:
  - Un IteradorConcreto guarda como pasa el objeto actual al siguiente del agregado.
- **Aplicabilidad**:
  - Para acceder a la información de un agregado sin exponer su estructura interna.
  - Se quiere permitir varios tipo de recorrido sobre un agregado.
  - Proporcionar una interfaz uniforme para recorrer diferentes tipos de agregados.

© JMA 2001. All rights reserved

Iterator

- **Consecuencias**:
  - Permite variaciones en el recorrido de un Agregado. Nuevos recorridos simplemente necesitan nuevas subclases de Iterador, si es una modificación solo es necesario cambiar la instancia de Iterador concreta
  - Puede hacerse más de un recorrido a la vez sobre un Agregado
  - Los Iteradores simplifican la interfaz del contenedor

© JMA 2001. All rights reserved

Iterator

- **Implementación**:
  - ¿Quién controla la iteración? Puede realizarse de forma externa a través del cliente o realizarlo de forma interna con el Iterador. En este caso, el Iterador recibe una función a aplicar sobre los elementos del Agregado y el recorrido es automático.
  - ¿Quién define el algoritmo de recorrido? Se puede hacer en el Iterador o en el Agregado. Si es en el caso del Agregado, se implementa el método siguiente y se deja el Iterador para almacenar la posición actual. En el caso de realizarlo en el Iterador, los recorridos son más fáciles de implementar pero se compromete la encapsulación.
  - ¿Cómo de robusto es el Iterador? Es recomendable formalizar un Iterador robusto que permita inserciones y borrados sin alterar el recorrido.
  - Se pueden añadir operaciones adicionales al Iterador.
  - Iteradores Nulos: Ayudan a manejar condiciones límite. El método Terminado() devuelve siempre True. Útil para estructuras heterogéneas

© JMA 2001. All rights reserved

Iterator

- **Usos conocidos** (Complejidad: media, Popularidad: alta):
  - La mayoría de los entornos modernos ya lo tienen implementado para proporcionar foreach y una forma estandarizada de recorrer sus colecciones.
  - Es muy utilizado en Java con java.util.Iterator y java.util.Enumeration.
- **Patrones relacionados**:
  - Composite: Iterador es a menudo utilizado como estructuras recursivas en Composites.
  - Memento: Es utilizado a menudo Memento para capturar el estado de una iteración.

© JMA 2001. All rights reserved

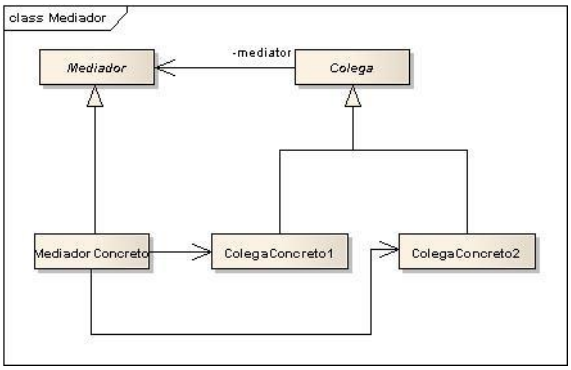
Mediator

- **Nombre**: Traducido como Mediador
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Un Mediador es un patrón de diseño que coordina las relaciones entre sus asociados. Define un objeto que encapsula cómo interactúan un conjunto de objetos. Promueve un bajo acoplamiento al evitar que los objetos se refieran unos a otros explícitamente, y permite variar la interacción entre ellos de forma independiente.
- **Motivación**:
  - Cuando muchos objetos interactúan con otros objetos, se puede formar una estructura muy compleja, con objetos con muchas conexiones con otros objetos. En un caso extremo cada objeto puede conocer a todos los demás objetos. Para evitar esto el patrón Mediator encapsula el comportamiento de todo un conjunto de objetos en un solo objeto. Los objetos envían y reciben peticiones a través del Mediador, este implementa el comportamiento cooperativo encaminando las peticiones a los objetos deseados.

© JMA 2001. All rights reserved

Mediator

Estructura:



Participantes:

- **Mediador**: Define una interfaz para comunicarse con los otros objetos
- **Colega**: Cada Colega conoce su Mediador, y usa a este para comunicarse con otros Colegas.
- **ColegaConcreto**: Implementa el comportamiento del Colega.
- **MediadorConcreto**: Implementa el comportamiento cooperativo entre los Colegas (como se comunican entre ellos). Además los conoce y mantiene.

© JMA 2001. All rights reserved

Mediator

- **Colaboraciones**:
  - Los Colegas envían y reciben requerimientos (requests) de un objeto Mediador, delegan en el Mediador. El Mediador implementa como se comunican los Colegas.
- **Aplicabilidad**:
  - Un conjunto grande de objetos se comunica de una forma bien definida, pero compleja.
  - Existe dificulta para reutilizar objetos, ya que nos referimos a varios objetos para comunicarnos.
  - El comportamiento de muchos objetos (que esta distribuido en varias clases) puede resumirse por subclasificación en una o varias.

© JMA 2001. All rights reserved

Mediator

- **Consecuencias**:
  - Desacopla a los Colegas: el patrón Mediador promueve bajar el acoplamiento entre Colegas. Se puede variar y rehusar Colegas y Mediadores independientemente.
  - Simplifica la comunicación entre objetos: Los objetos pueden remplazar una comunicación "muchos a muchos" por una del tipo "uno a muchos", que es menos compleja y más elegante. Además esta forma de comunicación es más fácil de entender.
  - Abstrae como los objetos cooperan: Haciendo que la mediación sea un concepto independiente y encapsulándolo en un objeto permite enfocar como los objetos interactúan. Esto ayuda a clarificar cómo los objetos se relacionan en un sistema.
  - Centraliza el control: El Mediador es el que se encarga de comunicar a los Colegas. Esto hace que el Mediador pueda ser muy complejo, difícil de entender y modificar.

© JMA 2001. All rights reserved

Mediator

- **Implementación**:
  - Omitir la clase abstracta Mediador: No es necesario crear una clase abstracta cuando los objetos solo trabajan con un Mediador. El acoplamiento abstracto de dicha clase permite que los objetos trabajen con diferentes subclases Mediador y viceversa.
  - Comunicación, Objeto y Mediador: Los objetos se comunican con su Mediador cuando se produce un evento. Cada vez que existe un cambio de estado lo notifican al Mediador. El Mediador responde propagando los efectos de dichos eventos a los objetos afectados.

© JMA 2001. All rights reserved

Mediator

- **Usos conocidos** (Complejidad: media, Popularidad: media):
  - Se suele emplear en gestores de cambio.
  - El uso más popular del patrón Mediator es para facilitar las comunicaciones entre los componentes GUI de una aplicación. El sinónimo del patrón Mediator es la parte del Controlador del patrón MVC.
- **Patrones relacionados**:
  - Fachada: Se diferencia en el uso de Fachada en que este abstrae un conjunto de objetos usando un protocolo unidireccional mientras que Mediador utiliza un protocolo multidireccional.
  - Observador: Mediante este patrón los objetos pueden comunicarse con el Mediador.

© JMA 2001. All rights reserved

Memento

- **Nombre**: También conocido como Recuerdo.
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - El patrón Memento guarda parte o todo el estado interno de un objeto, para que este estado pueda ser restaurado posteriormente. Esta operación debe ocurrir sin romper el principio del encapsulamiento.
- **Motivación**:
  - Muchas veces es necesario guardar el estado interno de un objeto. Esto es debido a que, tiempo después, se necesita restaurar el estado del objeto al que previamente se ha guardado. Consideremos, por ejemplo, una aplicación de composición de figuras geométricas, donde el usuario hace sucesivas modificaciones a una composición, pintando nuevas líneas, círculos y rectángulos. Después de cierto tiempo, el usuario logra una composición “casi perfecta”, pero decide alcanzar la perfección, así que pinta una línea y esta no le sale como él esperaba.

© JMA 2001. All rights reserved

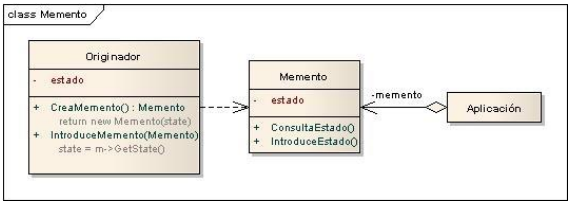
Memento

- **Motivación**:
  - Definitivamente el usuario querría regresar al instante en que su “creación” era una obra de arte. Para dar solución a este problema, antes de que el usuario agregue una nueva figura geométrica a la composición se debería guardar el estado de la composición y entonces siempre se tendría la posibilidad de regresar hacia atrás y restaurar la composición a su estado anterior.
  - Para lograr esto, sería necesario guardar la lista de figuras geométricas y el orden en que se encuentran en la composición, con información específica de cada una de ellas. En el caso de un círculo tendríamos que guardar la posición (x, y), el radio, color y relleno, para un rectángulo la posición (x, y), el ancho, el largo, color y relleno.
  - Para lograr esto tenemos tres alternativas: la primera alternativa consiste en obtener la lista de figuras de la composición y obtener su estado, esto sería muy complejo y además va en contra del principio de encapsulamiento; la segunda alternativa es que la composición se encargue de ir guardando su estado interno cada vez, esta no es una buena alternativa ya que la clase sería muy compleja y estaría asumiendo responsabilidades que no le corresponden; la tercera alternativa es la mejor: La composición crea un objeto (Memento) y almacena su estado interno en él, la aplicación mantiene una lista de los objetos (Memento), de tal manera que cuando el usuario realice una operación de “deshacer”, la aplicación restaura el estado de la composición con lo almacenado por el objeto Memento.

© JMA 2001. All rights reserved

Memento

Estructura:



Participantes:

- **Memento**: Almacena el estado de un objeto Originador.
  - Memento almacena todo o parte de Originador.
  - Tiene dos interfaces, una para Aplicación que le permite comunicarse con otros objetos y otra para Originador que le permite almacenar el estado
- **Originador**: Crea un objeto Memento con una copia de su estado. Usa Memento para restaurar el estado almacenado.
- **Aplicación**: Mantiene a Memento pero no opera con su contenido.

© JMA 2001. All rights reserved

Memento

- **Aplicabilidad**:
  - Todo o parte del objeto debe de ser almacenado para una posible restauración del mismo.
  - Cuando una interfaz directa para obtener el estado de un objeto exponga detalles de su implementación.
- **Consecuencias**:
  - Originador crea un Memento y él mismo almacena su estado interno, de esta manera no es necesario exponer el estado interno como atributos de acceso público, preservando así su encapsulación.
  - Si Originador tuviera que almacenar y mantener a salvo una o varias copias de su estado interno, sus responsabilidades crecerían y serían mas complejas, se desviaría de su propósito disminuyendo la coherencia.
  - Usar Mementos hace que el Originador sea mucho más sencillo y coherente. El uso frecuente de Mementos para almacenar estados internos de gran tamaño podría resultar costoso y perjudicar el rendimiento del sistema.
  - La Aplicación no conoce detalles del estado interno de Originador, no tiene idea de cuanto espacio y tiempo se necesita para almacenar el estado interno de Originador en un Memento y restaurar su estado interno a partir de un Memento, por lo que no puede hacer predicciones de tiempo ni de espacio.
  - Memento debe proporcionar una interfaz privada a la que solo Originador puede acceder. Esta interfaz incluye la creación, almacenamiento y recuperación del estado de Originador.

© JMA 2001. All rights reserved

Memento

- **Implementación**:
  - Memento crea una interfaz solo accesible por Originador.
  - Almacena los cambios incrementales: Todos los cambios de estado pueden almacenarse en Mementos.
- **Usos conocidos** (Complejidad: alta, Popularidad: baja):
  - Se utiliza en cualquier aplicación que necesite un "deshacer" como los editores de texto o los editores gráficos.
  - El principio del patrón Memento puede cumplirse utilizando la serialización, que es bastante habitual en Java o .NET. Aunque no es la única forma ni la más efectiva de realizar instantáneas del estado de un objeto, permite almacenar copias de seguridad del estado mientras protege de otros objetos la estructura del originador.
- **Patrones relacionados**:
  - Comando: Puede usar Mementos para restaurar sus operaciones almacenadas (undo).
  - Iterador: Puede usarse con Iterador para buscar colecciones de estados específicos.

© JMA 2001. All rights reserved

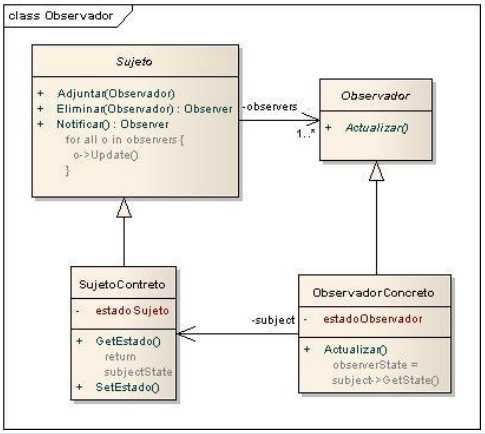
Observer

- **Nombre**: También conocido como Observador o Dependents
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Este patrón define una dependencia del tipo uno-a-muchos entre objetos, de manera que cuando uno de los objetos cambia su estado, el Observador se encarga de notificar este cambio a resto de los objetos dependientes. El objetivo de este patrón es desacoplar la clase de los objetos clientes del objeto, aumentando la modularidad del lenguaje, así como evitar bucles de actualización.
- **Motivación**:
  - Es necesario mantener la consistencia entre objetos relacionados sin aumentar el acoplamiento de las clases. Si imaginamos la relación entre la capa de presentación de un interfaz de usuario y los datos subyacentes en una representación gráfica observamos rápidamente qué pretende solucionar este patrón.
  - La interpretación en diferentes diagramas de un conjunto de datos depende de los datos de los objetos. Cualquier modificación en los mismos produce un efecto en la representación. El patrón observador establece las relaciones entre los cambios del sujeto y su efecto posterior.

© JMA 2001. All rights reserved

Observer

Estructura:



Participantes:

- **Observador**: Define la interfaz de los objetos a los que hay que notificar cambios del Sujeto.
- **ObservadorConcreto**: Mantiene una relación con un SujetoConcreto. Mantiene el estado del SujetoConcreto que le es de interés. Implementa Observador para mantener su estado coherente con el del Sujeto.
- **Sujeto**: Conoce a sus observadores. Tiene una interfaz para ir añadiendo y eliminando observadores.
- **SujetoConcreto**: Envía notificaciones a sus observadores cuando su estado cambia.

© JMA 2001. All rights reserved

Observer

- **Colaboraciones**:
  - SujetoConcreto notifica a sus observadores cualquier cambio que se produzca. Una vez notificado el cambio, el ObservadorConcreto puede notificar al Sujeto el mismo, actualizando el estado en el SujetoConcreto.
- **Aplicabilidad**:
  - Cuando una abstracción tiene dos aspectos dependientes el uno del otro. Encapsular los aspectos en objetos distintos permite cambiarlos y reutilizarlos.
  - Cuando cambiar un objeto implica cambiar otros pero no conocemos el número exacto a cambiar.
  - Cuando un objeto debe ser capaz de notificar algo a otros sin hacer suposiciones sobre quienes son dichos objetos. Es decir, cuando se quiere bajo acoplamiento.

© JMA 2001. All rights reserved

Observer

- **Consecuencias**:
  - Permite modificar Sujetos y Observadores de manera independiente.
  - Permite reutilizar un Sujeto sin reutilizar sus Observadores, y viceversa.
  - Permite añadir Observadores sin tener que cambiar el Sujeto ni demás Observadores.
  - Acoplamiento abstracto entre el Sujeto y el Observador. El Sujeto no sabe la clase concreta de sus Observadores.
  - Soporte para Broadcast. El Sujeto envía la notificación a todos los Observadores adscritos. Se pueden ir añadiendo o eliminando Observadores.
  - Actualizaciones inesperadas. Una operación puede provocar cambios en sus Observadores. El protocolo no ofrece detalles sobre lo que ha cambiado.

© JMA 2001. All rights reserved

Observer

- **Implementación**:
  - Correspondencia entre Sujetos y Observadores: Usualmente el Sujeto guarda una referencia a sus Observadores. En el caso de muchos Sujetos y pocos Observadores es recomendable el uso de una tabla Hash.
  - Observar más de un Sujeto: Es necesario extender la interfaz de actualización para el Observador sepa qué Sujeto manifestó el cambio
  - ¿Quién dispara la actualización mediante llamada a notificación? Se puede realizar de dos maneras. El Sujeto puede hacerlo desde los métodos que cambian de estado, lo que permite desacoplarlo del cliente pero es poco eficiente si hay varios cambios de estados consecutivos. También se puede llamar a la notificación desde el cliente, delegando en él la responsabilidad de la llamada a notificación
  - Referencias perdidas a Sujetos eliminados: Se pueden eliminar Sujetos si notificamos qué elementos se eliminan a sus Observadores. Asegurar la consistencia de un Sujeto previamente al envío de una notificación.
  - Simplificar las actualizaciones: Hacer que el método actualizar solo controle los aspectos del Sujeto que sean interesantes.

© JMA 2001. All rights reserved

Observer

- **Usos conocidos** (Complejidad: media, Popularidad: alta):
  - De uso común en la programación reactiva.
  - Es bastante habitual, sobre todo en los componentes GUI, frameworks de interfaces gráficas orientados a objetos. Proporciona una forma de reaccionar a los eventos que suceden en otros objetos, sin acoplarse a sus clases.
  - java.util.observer
- **Patrones relacionados**:
  - Singleton: El manejador de cambios puede usar el patrón Singleton para convertirlo en instancia única.

© JMA 2001. All rights reserved

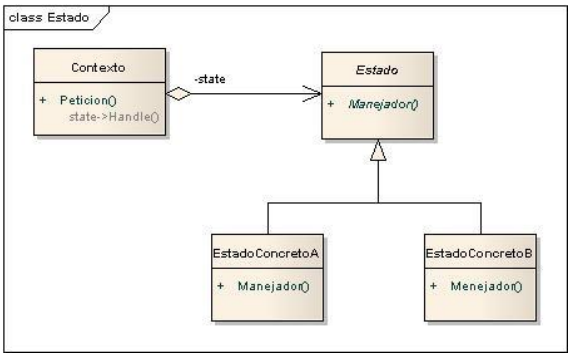
State

- **Nombre**: Traducido como Estado.
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Este patrón se utiliza cuando el comportamiento de un objeto cambia dependiendo del estado del mismo.
- **Motivación**:
  - Imaginemos la situación de algún objeto con varios estados en función de mensajes externos. Por ejemplo, una comunicación que usa el protocolo TCPConection, que representa una conexión de red. Un objeto de esta clase tendrá diferentes respuestas según su estado (Listening, Close o Established). Una llamada al método Open de un objeto de la clase TCPConection diferirá en su comportamiento si la conexión se encuentra en el estado Close o en el estado Established. La idea sería introducir una clase que, en función del estado y de los condicionantes que presenta el mismo, haga que el sistema responda adecuadamente. El patrón Estado proporciona una solución a esta problemática

© JMA 2001. All rights reserved

State

Estructura:



Participantes:

- **Contexto**: define la interfaz de interés para el cliente. Mantiene el una instancia EstadoConcreto con el estado actual
- **Estado**: define una interfaz para el encapsulamiento del comportamiento asociado a un estado particular del contexto
- **EstadoConcreto**: implementa el comportamiento de un estado de contexto.

© JMA 2001. All rights reserved

State

- **Colaboraciones**:
  - Contexto delega peticiones especificas de estado hacia el objeto de EstadoConcreto.
  - Contexto se envía a si mismo un argumento con el objeto estado que mantiene la petición
  - Contexto es la primera interfaz para el cliente
  - Tanto Contexto como EstadosConcretos pueden decidir cual es el siguiente estado en función de las condicionantes de la situación.
- **Aplicabilidad**:
  - El comportamiento de un objeto depende del estado en el que se encuentra y este estado se modifica en función de las transiciones de estado.
  - Si existen muchas operaciones con la misma lógica, el patrón permite introducir esta lógica en una clase separada.

© JMA 2001. All rights reserved

State

- **Consecuencias**:
  - Localiza el comportamiento dependiente del estado y divide dicho comportamiento en diferentes estados. Las transiciones de estados se reparten en las diferentes subclases.
  - Hace explicitas las transiciones entre estados. Introducir objetos separados para los diferentes estados hace que las transiciones sean más explicitas.
  - Los objetos Estado pueden compartirse.

© JMA 2001. All rights reserved

State

- **Implementación**:
  - ¿Quién dirige las transiciones? El patrón no indica exactamente dónde definir las transiciones de un estado a otro. Existen dos formas de solucionar esto: Una es definiendo estas transiciones dentro de la clase Contexto, la otra es definiendo estas transiciones en las subclases de Estado. Es más conveniente utilizar la primer solución cuando el criterio a aplicar es fijo, es decir, no se modificará. En cambio la segunda resulta conveniente cuando este criterio es dinámico, el inconveniente aquí se presenta en la dependencia de código entre las subclases.
  - Hay que evaluar en la implementación cuándo crear instancias de estado concreto distintas o utilizar la misma instancia compartida. Esto dependerá si el cambio de estado es menos o más frecuente respectivamente.
  - Usar la herencia dinámica permite a un objeto cambiar de clase en tiempo de ejecución dado que, al cambiar sus responsabilidades por las de otro objeto de otra clase, la herencia y responsabilidades del primero han cambiado por las del segundo.

© JMA 2001. All rights reserved

State

- **Usos conocidos** (Complejidad: baja, Popularidad: media):
  - Se utiliza habitualmente para convertir las enormes máquinas de estados, basadas en switch, en objetos.
  - Se usa para el protocolo conexión TCP.
  - Herramientas de dibujo.
- **Patrones relacionados**:
  - Singleton: puede utilizar el patrón Singleton cuando requiera controlar que exista una sola instancia de cada estado.
  - Flyweight: Lo puede utilizar cuando se comparten los objetos como Flyweight existiendo una sola instancia de cada estado y ésta es compartida con más de un objeto.

© JMA 2001. All rights reserved

Strategy

- **Nombre**: También conocido como Estrategia o Policy
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Es un patrón que permite mantener un conjunto de algoritmos de los que el objeto cliente puede elegir aquel que le conviene e intercambiarlo según sus necesidades. Los distintos algoritmos se encapsulan y el cliente trabaja contra un objeto Contexto. El cliente puede elegir el algoritmo que prefiera de entre los disponibles o puede ser el mismo objeto Contexto el que elija el más apropiado para cada situación.
  - Cualquier programa que ofrezca un servicio o función determinada, que pueda ser realizada de varias maneras, es candidato a utilizar el patrón Estrategia. Puede haber cualquier número de estrategias y cualquiera de ellas podrá ser intercambiada por otra en cualquier momento, incluso en tiempo de ejecución.

© JMA 2001. All rights reserved

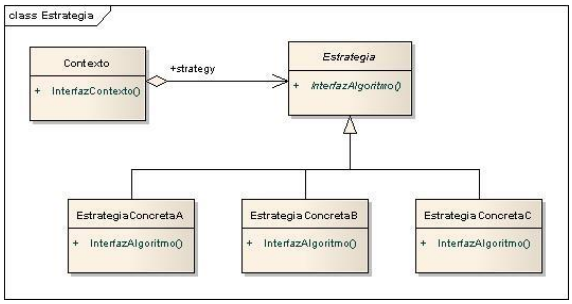
Strategy

- **Motivación**:
  - En muchas ocasiones una solución es valida solo para unas determinadas circunstancias. Si imaginamos el caso de un flujo de texto que queremos dividir en líneas podríamos solucionarlo codificando los algoritmos necesarios para cada posible situación en las clases afectadas. Pero esto lleva asociado las siguientes consecuencias:
    - Los clientes se hacen mas complejos
    - Distintos algoritmos serán apropiados en distintos momentos, hay un mantenimiento muy grande.
    - Es difícil añadir nuevos algoritmos y modificar los existentes
  - La mejor solución es definir clases que encapsulen los algoritmos y que sean llamadas por las clases que los necesitan, obteniendo así un desacoplamiento ante la aparición de nuevos algoritmos.

© JMA 2001. All rights reserved

Strategy

Estructura:



Participantes:

- **Contexto**: Esta configurado con un objeto de la EstrategiaConcreta. Mantienen una relación con el objeto Estrategia y define una interfaz para que Estrategia acceda a sus datos.
- **Estrategia**: Define una interfaz común a los algoritmos que soporta.
- **EstrategiaConcreta**: Define una implementación a un algoritmo mediante la interfaz de Estrategia.

© JMA 2001. All rights reserved

Strategy

- **Colaboraciones**:
  - Estrategia y Contexto interactúan para elegir el algoritmo adecuado. Contexto envía todos los datos necesarios para que Estrategia decida que algoritmo es llamado.
  - Contexto es la única interfaz que usan los clientes.
- **Aplicabilidad**:
  - Varias clases relacionadas que solo difieren en su comportamiento. Estrategia permite configurar a una clase con uno de los comportamientos.
  - Se necesitan variantes de un mismo algoritmo. Se implementan como una jerarquía de clases.
  - Un algoritmo usa datos que un cliente no debe conocer.
  - Una clase define muchos comportamientos basados en condicionales. Mover estas condiciones a la clases de Estrategia.

© JMA 2001. All rights reserved

Strategy

- **Consecuencias**:
  - Ayuda a eliminar sentencias condicionales.
  - Ayuda a factorizar la funcionalidad común de los algoritmos.
  - Hay mucha diversidad de estrategias para un comportamiento, por lo que el cliente debe comprender bien cómo funcionan cada una de ellas.
  - Es una alternativa a la subclases. Encapsulando los algoritmos en subclases separadas hace que dichos algoritmos sean totalmente independientes del contexto, facilitando la comprensión, entendimiento y extensión.
  - Se incrementa sustancialmente el numero de objetos.
  - Estimación "por lo alto" de las comunicaciones entre Estrategia y Contexto.

© JMA 2001. All rights reserved

Strategy

- **Implementación**:
  - Definición de la comunicación entre Contexto y Estrategia: Se puede optar por que el Contexto pase sus datos como argumento de las operaciones de la Estrategia (bajo acoplamiento, posible paso de parámetros innecesarios) o que el Contexto se pase a si mismo como argumento (alto acoplamiento)
  - Configurar Contexto con una Estrategia, si es posible usar la Estrategia en tiempo de compilación y no va a variar en tiempo de ejecución
  - Definir el comportamiento por defecto del Contexto en el caso de que no exista una Estrategia.

© JMA 2001. All rights reserved

Strategy

- **Usos conocidos** (Complejidad: baja, Popularidad: alta);
  - Se utiliza a menudo en varios frameworks para proporcionar a los usuarios una forma de cambiar el comportamiento de una clase sin extenderla.
  - El soporte de funciones lambda, que pueden servir como alternativas más sencillas al patrón Strategy.
- **Patrones relacionados**:
  - Template Method: Tiene una intención similar pero haciendo uso de la sobre escritura en lugar de la delegación.

© JMA 2001. All rights reserved

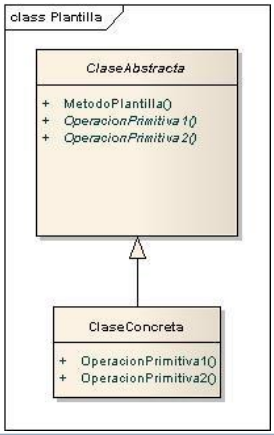
Template Method

- **Nombre**: Traducido como Métodos Plantilla.
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Es un patrón de diseño que define una estructura algorítmica en una súper clase, delegando la implementación a las subclases. Es decir, define un proceso que utiliza una serie de pasos abstractos que serán implementados en las subclases.
- **Motivación**:
  - Imaginemos una situación donde manejamos una aplicación que accede a documentación. La clase que mantiene la aplicación será la responsable de abrir los documentos, mientras que una clase Documentación será la encargada de mantener la información de los mismos. Cada aplicación tendrá su clase de Documentación asociada. En definitiva, vamos construyendo métodos, denominados Métodos Plantilla, que incluyen en su código llamadas a métodos abstractos. El concepto de plantilla consiste en definir una estructura que se rellenará mas adelante, con las implementaciones de sus herederos.

© JMA 2001. All rights reserved

Template Method

Estructura:



Participantes:

- **ClaseAbstracta**: Implementa un método plantilla, que define el esqueleto de un algoritmo, y define los métodos abstractos utilizados en el algoritmo, que implementas las ClasesConcretas.
- **ClaseConcreta**: Implementa los métodos abstractos para realizar los pasos del algoritmo que son específicos de la subclase.

© JMA 2001. All rights reserved

Template Method

- **Colaboraciones**:
  - Las ClasesConcretas confían en que las ClasesAbstractas implemente la parte fija del algoritmo.
- **Aplicabilidad**:
  - Factorizar el comportamiento común de varias subclases.
  - Implementar las partes fijas de una algoritmo una sola vez y dejar que las subclases implementen las partes variables.
  - Cuando se diseña una biblioteca. Algunas de las clases pueden tener un comportamiento que dependa de la aplicación que la use. En tal caso, este patrón obliga a que el programador proporcione dicho comportamiento.
  - Definir una estructura lo más reutilizable posible para la autentificación de usuarios.
  - Controlar las ampliaciones de las subclases, convirtiendo en métodos plantillas aquellos métodos que pueden ser redefinidos.

© JMA 2001. All rights reserved

Template Method

- **Consecuencias**:
  - Favorece la reutilización de código: Muy útil para construir bibliotecas, pues ayuda a factorizar el comportamiento común de las clases.
  - Lleva a una estructura de control invertido: La superclase base invoca los métodos de las subclases.
  - Delegación por herencia.
- **Implementación**:
  - No redefinir los métodos plantilla.
  - Los métodos abstractos deben ser protegidos (accesible a las subclases pero no a los clientes) y abstractos.
  - Minimizar el número de métodos abstractos a fin de facilitar la implementación de las subclases.
  - Convenios de denominación: Se suele usar el prefijo DO.

© JMA 2001. All rights reserved

Template Method

- **Usos conocidos** (Complejidad: baja, Popularidad: media):
  - Se suelen encontrar en todas las clases abstractas
  - Es muy común en los frameworks, se utiliza a menudo para proporcionar medios sencillos para extender la funcionalidad estándar utilizando la herencia.
- **Patrones relacionados**:
  - Estrategia: Usa la composición para cambiar todo el algoritmo, los Métodos Plantilla usan la herencia para cambiar parte de un algoritmo.
  - Métodos Factoría: Los métodos factoría suelen llamarse desde Métodos Plantilla.

© JMA 2001. All rights reserved

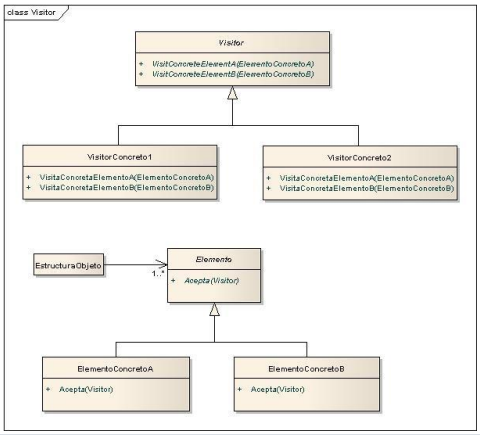
Visitor

- **Nombre**: Traducido como Visitante
- **Clasificación**: Patrón de comportamiento
- **Descripción**:
  - Es un patrón que permite definir nuevas operaciones sobre una jerarquía de clases sin modificar las clases sobre las que opera.
- **Motivación**:
  - Consideremos una aplicación relacionada con compiladores. Estos representan los programa en árboles de sintaxis abstracta sobre los que ejecutan operaciones para realizar su funcionalidad. Muchas de estas operaciones tienen un contexto propio y por lo tanto necesitan diferenciar cierta información (tipo de nodo del árbol). Esto genera un conflicto, ya que lo convierte en difícil de implementar y mantener. La aparición de nuevos elementos supondría recompilar todas las clases existentes.
  - El patrón Visitor propone una solución a este problema: Pretende independizar las clases de las operaciones que se ejecutan sobre ellas.

© JMA 2001. All rights reserved

Visitor

Estructura:



Participantes:

- **Visitor**: Define una operación de visita para cada clase de elemento concreto en la estructura de objetos
- **VisitorConcreto**: Implementa la interfaz de Visitor. Cada operación implementa un fragmento de la labor global del VisitorConcreto pudiendo almacenar información local
- **Elemento**: Define la operación Acepta, que recibe un Visitor, sobre el que ejecuta la visita
- **ElementoConcreto**: implementa la operación Acepta
- **EstructuraObjeto**: Gestiona la estructura de objetos y puede enumerar sus elementos. Puede ser un compuesto (Composite) o una colección de objetos. Puede ofrecer una interfaz que permita visitar a sus elementos al Visitor

© JMA 2001. All rights reserved

Visitor

- **Aplicabilidad**:
  - Una estructura de objetos contiene muchas clases de objetos con interfaces distintas y se quiere realizar sobre ellos operaciones que son distintas en cada clase concreta.
  - Se quieren realizar muchas operaciones distintas sobre los objetos de una estructura, sin incluir dichas operaciones en las clases.
  - Las clases que forman la estructura de objetos no cambian pero las operaciones sobre ellas sí.
  - Se utiliza cuando es necesario realizar una operación sobre todos los elementos de una compleja estructura de objetos (por ejemplo, un árbol de objetos).
  - Permite limpiar la lógica de negocio de comportamientos auxiliares.

© JMA 2001. All rights reserved

Visitor

- **Consecuencias**:
  - Facilita la definición de nuevas operaciones.
  - Agrupa operaciones relacionadas.
  - Añadir nuevos ElementosConcretos es costoso. Solo se debe utilizar el patrón si la jerarquía es estable.
  - Permite atravesar jerarquías de objetos que no están relacionadas por un padre común.
  - Puede acumular el estado de una operación al visitar la estructura de objetos, en vez de pasarlo como argumento o variable global.
  - Rompe la encapsulación.

© JMA 2001. All rights reserved

Visitor

- **Implementación**:
  - Double Dispatch: Técnica que permite añadir operaciones a las clases sin tener que modificarlas. La operación a ejecutar depende de la clase de petición (acepta) y del tipo de los dos receptores (Visitor, Elemento)
  - ¿Quién es responsable de recorrer la estructura de objetos? Un Visitor debe de recorrer cada elemento de la estructura. Podemos hacerlo mediante un objeto de la estructura, un objeto Visitor o mediante un iterador. Si lo hacemos mediante un Visitor se duplica el código recorrido en cada objeto de tipo compuesto. Sólo se utiliza para implementar recorridos complejos que dependen de los resultados de las operaciones.

© JMA 2001. All rights reserved

Visitor

- **Usos conocidos (Complejidad: alta, Popularidad: baja):**
  - El patrón Visitor no es muy habitual debido a su complejidad y limitada aplicabilidad.
  - Este patrón es ampliamente utilizado en intérpretes, compiladores y procesadores de lenguajes, en general.
- **Patrones relacionados:**
  - Puedes tratar a Visitor como una versión potente del patrón Command. Sus objetos pueden ejecutar operaciones sobre varios objetos de distintas clases.
  - Puedes utilizar el patrón Visitor para ejecutar una operación sobre un árbol Composite entero.
  - Los Visitor pueden ser aplicados para realizar la interpretación

© JMA 2001. All rights reserved

PATRONES ARQUITECTÓNICOS

© JMA 2001. All rights reserved

Otros patrones

- Arquitectura Concéntricas:
  - Hexagonal, Onion y Clean Architecture
- Patrones DDD:
  - Entity, Value Object, DTO, Agregate, Repository, Service, …
- Patrones de Inversión de Control e Inyección de Dependencias
- Patrones de Presentación:
  - MVC, MVP y MVVM, Composite View
- Patrón de testing AAA

© JMA 2001. All rights reserved